MODUL 09 · REST API & HTTP

Integrasi REST API & Asynchronous Networking

Menghubungkan aplikasi Flutter ke server backend melalui HTTP/REST secara bertahap: konsep request-response, JSON dan model Dart (fromJson/toJson), GET/POST/PUT/PATCH/DELETE dengan package http, header & Bearer token, penanganan error dan timeout, lapisan ApiClient-Repository, FutureBuilder, pull-to-refresh, integrasi dengan controller GetX (pola MVC), hingga studi kasus aplikasi CRUD Posts multi-file.

Durasi Pertemuan 9 (150 Menit) + Belajar Mandiri

Prasyarat Modul 08: State Management GetX & Pola MVC

Capaian Pembelajaran

- Menjelaskan komponen request/response HTTP: method, URL, header, body, dan status code.
- Mengonversi JSON menjadi objek Dart (jsonDecode, fromJson) termasuk object bersarang dan list, serta sebaliknya (toJson, jsonEncode).
- Melakukan HTTP Request (GET, POST, PUT, PATCH, DELETE) menggunakan package http dan mengirim header Authorization Bearer.
- Menangani kegagalan jaringan (status code, SocketException, ClientException, TimeoutException) dengan pesan yang ramah pengguna.
- Menyusun lapisan ApiClient dan Repository sehingga UI terpisah dari kode jaringan.
- Menampilkan tiga kondisi UI (loading, error, data) dengan FutureBuilder maupun controller GetX (isLoading, error, data), dilengkapi pull-to-refresh.
- Membangun aplikasi CRUD lengkap (daftar, detail, tambah, edit, hapus) berpola MVC dengan GetX yang terhubung ke REST API.

Daftar Isi Modul

1. Konsep HTTP & REST: Method, Status Code, Header, Body
2. Dasar JSON: jsonDecode & jsonEncode
3. Model Dart: fromJson & toJson (Nested & List)
4. Persiapan: Package http & Izin Internet Android/iOS
5. GET: Mengambil Daftar dan Detail Data
6. POST, PUT, PATCH, dan DELETE
7. Header & Bearer Token
8. Penanganan Error, Timeout, dan Offline
9. Service Layer: ApiClient & Repository
10. Menangani 3 Kondisi UI dengan FutureBuilder
11. Pull-to-Refresh dengan RefreshIndicator

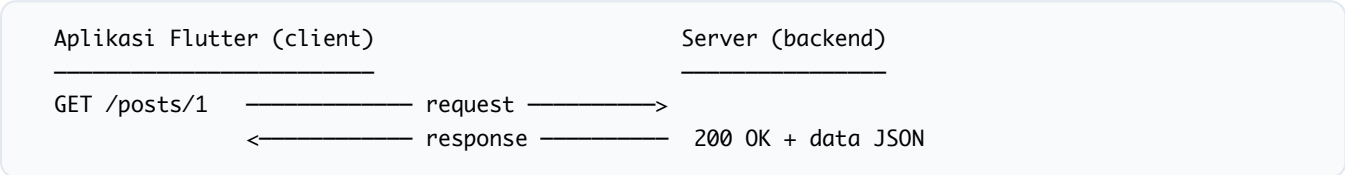
12. Integrasi REST API dengan GetX (Controller & View)
13. Studi Kasus: Aplikasi CRUD Posts Multi-File
14. Tugas Praktikum
15. Kuis Pemahaman & Kunci Jawaban

1. Konsep HTTP & REST: Method, Status Code, Header, Body

Hampir semua aplikasi modern (Gojek, Tokopedia, SIA kampus) tidak menyimpan seluruh datanya di HP. Data diambil dari **server** lewat internet. Cara paling umum untuk "mengobrol" dengan server adalah **REST API** yang berjalan di atas protokol **HTTP**. Sebelum menulis kode Flutter, pahami dulu bahasa percakapannya.

Client, Server, Request, Response

Aplikasi Flutter berperan sebagai **client**. Client mengirim **request** (permintaan) ke **server**, lalu server membalas dengan **response** (jawaban). Satu request selalu dibalas satu response.

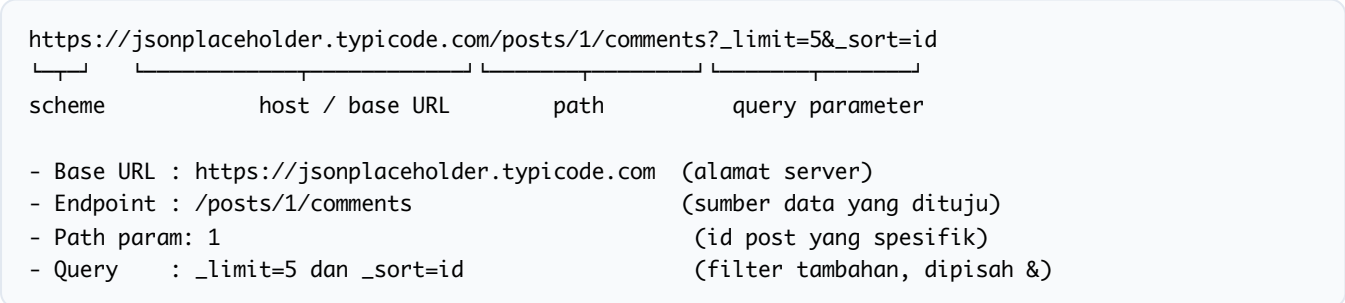


Setiap request terdiri dari 4 bagian utama:

1. **Method**: apa yang ingin dilakukan (ambil, buat, ubah, hapus).
2. **URL**: alamat data yang dituju.
3. **Header**: informasi tambahan (format data, token login).
4. **Body**: isi data yang dikirim (biasanya JSON). Request GET dan DELETE umumnya tidak punya body.

Response juga punya **status code** (angka hasil), **header**, dan **body**.

Anatomi URL



HTTP Method (Kata Kerja REST)

REST memakai kata benda untuk URL (`/posts`, `/users`) dan method HTTP sebagai kata kerjanya:

Method	Fungsi	Contoh	Ada body?	Padanan CRUD
`GET`	Mengambil data	`GET /posts`, `GET /posts/1`	Tidak	Read
`POST`	Membuat data baru	`POST /posts`	Ya	Create
`PUT`	Mengganti *seluruh* data	`PUT /posts/1`	Ya (lengkap)	Update
`PATCH`	Mengubah *sebagian* field	`PATCH /posts/1`	Ya (sebagian)	Update
`DELETE`	Menghapus data	`DELETE /posts/1`	Tidak	Delete

Status Code

Status code adalah angka 3 digit yang memberi tahu hasil request. Digit pertama menunjukkan kelompoknya:

Kode	Nama	Arti untuk aplikasi
`200`	OK	Berhasil (GET, PUT, PATCH, DELETE).
`201`	Created	Data baru berhasil dibuat (POST).
`204`	No Content	Berhasil, tetapi body kosong (sering untuk DELETE).
`400`	Bad Request	Data yang dikirim salah format / tidak valid.
`401`	Unauthorized	Belum login atau token salah/kedaluwarsa.
`403`	Forbidden	Sudah login, tetapi tidak punya hak akses.
`404`	Not Found	URL atau data dengan id tersebut tidak ada.
`422`	Unprocessable Entity	Validasi gagal (mis. email sudah dipakai).
`500`	Internal Server Error	Server sedang bermasalah, bukan salah aplikasi.
`503`	Service Unavailable	Server sedang down / maintenance.

> 💡 **Cara Mudah Mengingat**
>
> **2xx** = sukses, **4xx** = kesalahan dari sisi client (aplikasi kita), **5xx** = kesalahan dari sisi server. Di kode, cek sukses dengan `statusCode >= 200 && statusCode < 300`.

Header yang Sering Dipakai

Header	Contoh nilai	Kegunaan
`Content-Type`	`application/json; charset=UTF-8`	Memberi tahu server bahwa body yang kita kirim berformat JSON.
`Accept`	`application/json`	Meminta server membalas dalam format JSON.
`Authorization`	`Bearer eyJhbGciOi...`	Mengirim token login agar server tahu siapa kita.
`x-api-key`	`abc123`	Kunci API khusus yang diminta sebagian layanan.

Contoh Request & Response Lengkap

Beginilah wujud "mentah" satu request POST dan response-nya. Package `http` di Flutter akan membuat teks seperti ini untuk kita.

```
POST /posts HTTP/1.1
Host: jsonplaceholder.typicode.com
Content-Type: application/json; charset=UTF-8
Accept: application/json

{"title": "Belajar REST", "body": "Isi tulisan", "userId": 1}

------------------------------------------------------------

HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{"title": "Belajar REST", "body": "Isi tulisan", "userId": 1, "id": 101}
```

Contoh Kode: Menyusun URL dengan Kelas `Uri`

Di Dart, URL disimpan sebagai objek `Uri`. Ada dua cara membuatnya. `Uri.parse` cocok untuk URL sederhana, sedangkan `Uri.https` lebih aman untuk query parameter karena karakter khusus (spasi, &) otomatis di-*encode*.

`lib/contoh_uri.dart`

```dart
// Jalankan dengan: dart run lib/contoh_uri.dart
void main() {
  // Cara 1: Uri.parse dari string utuh
  final uri1 = Uri.parse('https://jsonplaceholder.typicode.com/posts/1');
  print(uri1.host); // jsonplaceholder.typicode.com
  print(uri1.path); // /posts/1

  // Cara 2: Uri.https(host, path, queryParameters)
  final uri2 = Uri.https(
    'jsonplaceholder.typicode.com',
    '/posts',
    {'userId': '1', '_limit': '5'}, // nilai query WAJIB String
  );
  print(uri2); // https://jsonplaceholder.typicode.com/posts?userId=1&_limit=5

  // Karakter khusus otomatis di-encode
  final uri3 = Uri.https('example.com', '/cari', {'q': 'flutter & dart'});
  print(uri3); // https://example.com/cari?q=flutter+%26+dart

  // Menambah query ke Uri yang sudah ada
  final uri4 = uri1.replace(queryParameters: {'lang': 'id'});
  print(uri4); // https://jsonplaceholder.typicode.com/posts/1?lang=id
}
```

Penjelasan: nilai di `queryParameters` harus bertipe `String`, jadi angka ditulis `'1'` bukan `1`. Bagian `uri3` memperlihatkan alasan memakai `Uri.https`: tanda `&` di dalam kata kunci pencarian berubah menjadi `%26` sehingga tidak dianggap pemisah query.

> ⚠️ **Kesalahan Umum**
>
> - Menulis host dengan `https://` di `Uri.https('https://example.com', ...)`. Argumen pertama hanya host, tanpa scheme.
> - Mengirim angka di `queryParameters` (`{'page': 1}`) sehingga error tipe. Ubah menjadi `'1'` atau `1.toString()`.
> - Menganggap status `404` akan melempar exception. Package `http` **tidak** melempar error untuk 4xx/5xx; kita sendiri yang wajib mengecek `statusCode`.
> - Memakai GET untuk mengirim data sensitif lewat query (mis. password). Data di URL tercatat di log server; kirim lewat body POST.

2. Dasar JSON: jsonDecode & jsonEncode

Server mengirim data dalam bentuk **teks JSON**. Teks ini harus diubah dulu menjadi objek Dart (*decode*) agar bisa dipakai, dan objek Dart harus diubah menjadi teks JSON (*encode*) sebelum dikirim ke server. Keduanya disediakan library bawaan `dart:convert`, tanpa perlu package tambahan.

Wujud JSON

JSON hanya mengenal 6 jenis nilai. Hafalkan padanannya di Dart karena ini sumber error paling sering:

JSON	Contoh	Hasil di Dart setelah `jsonDecode`
Object	`{"nama": "Budi"}`	`Map<String, dynamic>`
Array	`[1, 2, 3]`	`List<dynamic>` (bukan `List<int>`!)
String	`"halo"`	`String`
Number	`10` / `3.5`	`int` / `double`
Boolean	`true`	`bool`
null	`null`	`null`

Contoh JSON satu mahasiswa yang berisi object bersarang (`prodi`) dan array (`hobi`):

```
{
  "nim": "H1D022001",
  "nama": "Budi Santoso",
  "ipk": 3.75,
  "aktif": true,
  "prodi": { "kode": "IF", "nama": "Informatika" },
  "hobi": ["coding", "futsal"]
}
```

Contoh Kecil: `jsonDecode` dan `jsonEncode`

```dart
import 'dart:convert';

void main() {
  // Teks JSON (String) -> objek Dart
  const teks = '{"nama": "Budi", "umur": 20}';
  final Map<String, dynamic> data = jsonDecode(teks);
  print(data['nama']); // Budi
```

```
  // Objek Dart -> teks JSON (String)
  final kirim = {'nama': 'Siti', 'umur': 21};
  final String json = jsonEncode(kirim);
  print(json); // {"nama":"Siti","umur":21}
}
```

`jsonDecode` menerima `String` dan mengembalikan `dynamic`. Karena itu kita menyebutkan tipe tujuannya (`Map<String, dynamic>`) agar editor bisa membantu.

Contoh Lengkap: Membaca Data Bersarang dan Array

`lib/contoh_json.dart`

```
// Jalankan dengan: dart run lib/contoh_json.dart
import 'dart:convert';

const String responseBody = '''
{
  "nim": "H1D022001",
  "nama": "Budi Santoso",
  "ipk": 3.75,
  "aktif": true,
  "prodi": { "kode": "IF", "nama": "Informatika" },
  "hobi": ["coding", "futsal"],
  "nilai": [
    { "matkul": "Pemrograman Mobile", "huruf": "A" },
    { "matkul": "Basis Data", "huruf": "AB" }
  ]
}
''';

void main() {
  // 1. Decode teks menjadi Map
  final Map<String, dynamic> mhs = jsonDecode(responseBody) as Map<String, dynamic>;

  // 2. Ambil nilai sederhana dengan cast "as"
  final String nama = mhs['nama'] as String;
  final double ipk = (mhs['ipk'] as num).toDouble(); // aman untuk 3 maupun 3.75
  final bool aktif = mhs['aktif'] as bool;
  print('$nama, IPK $ipk, aktif: $aktif');

  // 3. Object bersarang adalah Map di dalam Map
  final Map<String, dynamic> prodi = mhs['prodi'] as Map<String, dynamic>;
  print('Prodi: ${prodi['nama']}');

  // 4. Array berisi String: List<dynamic> harus dikonversi
  final List<String> hobi = List<String>.from(mhs['hobi'] as List);
  print('Hobi: ${hobi.join(', ')}');

  // 5. Array berisi object: tiap elemen adalah Map
  final List<dynamic> nilaiList = mhs['nilai'] as List<dynamic>;
  for (final item in nilaiList) {
    final nilai = item as Map<String, dynamic>;
    print('- ${nilai['matkul']}: ${nilai['huruf']}');
  }

  // 6. Field yang tidak ada menghasilkan null, gunakan ?? untuk nilai default
```

```
  final String? email = mhs['email'] as String?;
  print('Email: ${email ?? 'belum diisi'}');

  // 7. Encode kembali; JsonEncoder.withIndent membuat output rapi
  mhs['aktif'] = false;
  const encoder = JsonEncoder.withIndent('  ');
  print(encoder.convert(mhs));
}
```

Penjelasan baris penting:

- **Langkah 2**: angka JSON `3` menjadi `int`, sedangkan `3.75` menjadi `double`. Menulis `as double` akan crash jika server mengirim `3`. Pola `(x as num).toDouble()` aman untuk keduanya.
- **Langkah 4**: hasil decode array selalu `List<dynamic>`. `List<String>.from(...)` menyalin isinya ke list bertipe benar.
- **Langkah 6**: kunci yang tidak ada di Map menghasilkan `null`, bukan error. Pakai tipe nullable (`String?`) lalu beri nilai default.

Mengirim Data: Map Dart ke Body JSON

Saat mengirim POST/PUT, body harus berupa String JSON. Objek seperti `DateTime` tidak bisa langsung di-encode, jadi ubah dulu menjadi String:

```
import 'dart:convert';

void main() {
  final sekarang = DateTime(2026, 9, 1, 8, 30);

  final body = jsonEncode({
    'judul': 'Kuis 1',
    'tanggal': sekarang.toIso8601String(), // DateTime -> "2026-09-01T08:30:00.000"
    'peserta': ['Budi', 'Siti'],
    'selesai': false,
  });
  print(body);

  // Membaca kembali tanggal dari JSON
  final data = jsonDecode(body) as Map<String, dynamic>;
  final tanggal = DateTime.parse(data['tanggal'] as String);
  print(tanggal.day); // 1
}
```

⚠️ Kesalahan Umum

- `type 'List<dynamic>' is not a subtype of type 'List<String>'` : menulis `mhs['hobi'] as List<String>`. Gunakan `List<String>.from(...)` atau `.cast<String>()`.
- `type 'int' is not a subtype of type 'double'` : lihat langkah 2, pakai `num`.
- `FormatException: Unexpected character` : teks yang di-decode bukan JSON, biasanya halaman HTML error dari server. Cek `statusCode` sebelum decode.
- `Converting object to an encodable object failed: Instance of 'DateTime'` : encode objek yang bukan tipe JSON. Ubah ke String/Map dulu.
- Salah huruf kunci: `data['Nama']` berbeda dengan `data['nama']`. Kunci JSON *case-sensitive*. Inilah alasan kita membuat Model Class di section berikutnya.

3. Model Dart: fromJson & toJson (Nested & List)

Jangan pernah menggunakan raw `Map<String, dynamic>` langsung di widget UI karena rentan salah ketik kunci string (typo) dan runtime error. Selalu petakan respon API ke dalam **Model Class** berstruktur kuat. Dengan model, editor akan memberi saran otomatis (*autocomplete*) dan kesalahan nama field langsung terdeteksi sebelum aplikasi dijalankan.

Pola Dasar: `fromJson` dan `toJson`

- `factory NamaModel.fromJson(Map<String, dynamic> json)`: mengubah Map hasil `jsonDecode` menjadi objek.
- `Map<String, dynamic> toJson()`: mengubah objek menjadi Map yang siap di-`jsonEncode`.

Contoh model mahasiswa sederhana:

`lib/models/mahasiswa_model.dart`

```dart
// lib/models/mahasiswa_model.dart
// Model Data Mahasiswa Berbasis JSON
class MahasiswaModel {
  final int id;
  final String nama;
  final String email;
  final String nim;
  final String avatarUrl;

  MahasiswaModel({
    required this.id,
    required this.nama,
    required this.email,
    required this.nim,
    required this.avatarUrl,
  });

  // Factory Constructor untuk mengurai JSON dari API Server
  factory MahasiswaModel.fromJson(Map<String, dynamic> json) {
    return MahasiswaModel(
      id: json['id'] as int,
      nama: json['nama'] as String? ?? 'Mahasiswa Anonim',
      email: json['email'] as String? ?? '-',
      nim: json['nim'] as String? ?? '-',
      avatarUrl: json['avatar'] as String? ?? 'https://i.pravatar.cc/150',
    );
  }

  // Method untuk mengirim data objek kembali ke format JSON payload
  Map<String, dynamic> toJson() {
    return {
      'nama': nama,
      'email': email,
      'nim': nim,
    };
  }
}
```

Penjelasan: `json['nama'] as String? ?? 'Mahasiswa Anonim'` dibaca: "ambil `nama`, boleh null, jika null pakai nilai default". Pola ini membuat aplikasi tidak crash ketika server lupa mengirim satu field. Perhatikan juga `toJson()` tidak menyertakan `id` karena id biasanya dibuat oleh server.

Model untuk JSONPlaceholder: `Post`

Sepanjang modul ini kita memakai API publik gratis jsonplaceholder.typicode.com. Satu post di sana berbentuk:

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident",
  "body": "quia et suscipit suscipit recusandae..."
}
```

lib/models/post.dart

```dart
class Post {
  final int? id; // null ketika post belum dikirim ke server
  final int userId;
  final String title;
  final String body;

  const Post({
    this.id,
    required this.userId,
    required this.title,
    required this.body,
  });

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'] as int?,
      userId: json['userId'] as int? ?? 0,
      title: json['title'] as String? ?? '',
      body: json['body'] as String? ?? '',
    );
  }

  Map<String, dynamic> toJson() {
    return {
      if (id != null) 'id': id, // "collection if": id hanya ditulis jika ada
      'userId': userId,
      'title': title,
      'body': body,
    };
  }

  // copyWith: membuat salinan dengan sebagian field diganti (berguna saat edit)
  Post copyWith({int? id, int? userId, String? title, String? body}) {
    return Post(
      id: id ?? this.id,
      userId: userId ?? this.userId,
      title: title ?? this.title,
      body: body ?? this.body,
```

```
    );
  }

  @override
  String toString() => 'Post(id: $id, title: $title)';
}
```

Mengubah Array JSON Menjadi `List<Post>`

Endpoint `/posts` mengembalikan array. Polanya selalu sama: decode menjadi `List`, lalu `map` setiap elemen ke `fromJson`.

```
List<Post> parsePosts(String responseBody) {
  final List<dynamic> data = jsonDecode(responseBody) as List<dynamic>;
  return data
      .map((item) => Post.fromJson(item as Map<String, dynamic>))
      .toList();
}
```

Object Bersarang (Nested) dan List di Dalam Model

Endpoint `/users` berisi object di dalam object (`address` berisi `geo`, serta `company`). Buat **satu class untuk setiap object**, lalu panggil `fromJson` milik class anak dari dalam class induk.

```
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "city": "Gwenborough",
    "geo": { "lat": "-37.3159", "lng": "81.1496" }
  },
  "company": { "name": "Romaguera-Crona" }
}
```

`lib/models/user.dart`

```
class Geo {
  final double lat;
  final double lng;

  const Geo({required this.lat, required this.lng});

  // API mengirim koordinat sebagai String, jadi kita parse ke double
  factory Geo.fromJson(Map<String, dynamic> json) {
    return Geo(
      lat: double.tryParse(json['lat']?.toString() ?? '') ?? 0,
      lng: double.tryParse(json['lng']?.toString() ?? '') ?? 0,
    );
  }

  Map<String, dynamic> toJson() => {'lat': lat.toString(), 'lng': lng.toString()};
}
```

```dart
class Address {
  final String street;
  final String city;
  final Geo geo;

  const Address({required this.street, required this.city, required this.geo});

  factory Address.fromJson(Map<String, dynamic> json) {
    return Address(
      street: json['street'] as String? ?? '',
      city: json['city'] as String? ?? '',
      geo: Geo.fromJson(json['geo'] as Map<String, dynamic>? ?? {}),
    );
  }

  Map<String, dynamic> toJson() => {
        'street': street,
        'city': city,
        'geo': geo.toJson(), // anak juga di-toJson
      };
}

class User {
  final int id;
  final String name;
  final String email;
  final Address address;
  final String companyName;
  final List<String> roles; // contoh list String

  const User({
    required this.id,
    required this.name,
    required this.email,
    required this.address,
    required this.companyName,
    this.roles = const [],
  });

  factory User.fromJson(Map<String, dynamic> json) {
    final company = json['company'] as Map<String, dynamic>? ?? {};
    return User(
      id: json['id'] as int,
      name: json['name'] as String? ?? '',
      email: json['email'] as String? ?? '',
      address: Address.fromJson(json['address'] as Map<String, dynamic>),
      companyName: company['name'] as String? ?? '-',
      roles: List<String>.from(json['roles'] as List? ?? const []),
    );
  }

  Map<String, dynamic> toJson() => {
        'id': id,
        'name': name,
        'email': email,
        'address': address.toJson(),
        'company': {'name': companyName},
        'roles': roles,
```

```
      };
}
```

Contoh Lengkap yang Bisa Dijalankan

Program berikut memakai model `Post` dan `User` di atas dengan data JSON contoh (belum memakai internet), supaya kamu bisa fokus pada proses parsing.

lib/contoh_model.dart

```dart
// Jalankan dengan: dart run lib/contoh_model.dart
import 'dart:convert';

import 'models/post.dart';
import 'models/user.dart';

const postsJson = '''
[
  {"userId": 1, "id": 1, "title": "Halo Flutter", "body": "Belajar REST API"},
  {"userId": 1, "id": 2, "title": "JSON itu mudah", "body": "Asal pakai model"}
]
''';

const userJson = '''
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "city": "Gwenborough",
    "geo": {"lat": "-37.3159", "lng": "81.1496"}
  },
  "company": {"name": "Romaguera-Crona"},
  "roles": ["admin", "editor"]
}
''';

void main() {
  // Array -> List<Post>
  final List<dynamic> rawPosts = jsonDecode(postsJson) as List<dynamic>;
  final posts = rawPosts
      .map((e) => Post.fromJson(e as Map<String, dynamic>))
      .toList();
  for (final post in posts) {
    print('${post.id}. ${post.title}');
  }

  // Object bersarang -> User
  final user = User.fromJson(jsonDecode(userJson) as Map<String, dynamic>);
  print('${user.name} tinggal di ${user.address.city}');
  print('Koordinat: ${user.address.geo.lat}, ${user.address.geo.lng}');
  print('Peran: ${user.roles.join(', ')}');

  // Model -> JSON (untuk dikirim ke server)
  final baru = Post(userId: 1, title: 'Post baru', body: 'Isi');
  print(jsonEncode(baru.toJson())); // tanpa "id"
```

```dart
  // copyWith untuk edit
  final diedit = posts.first.copyWith(title: 'Judul diganti');
  print(diedit); // Post(id: 1, title: Judul diganti)
}
```

> 💡 **Tips: Generator Otomatis**
>
> Untuk model yang sangat besar, ada package `json_serializable` yang membuat `fromJson` / `toJson` otomatis. Namun pahami dulu cara manual seperti di atas, karena hasil generator pun bentuknya sama.

> ⚠️ **Kesalahan Umum**
>
> - Lupa memanggil `Address.fromJson(...)` sehingga field bertipe `Address` diisi Map, hasilnya error tipe.
> - Lupa memanggil `address.toJson()` di dalam `toJson()` induk sehingga `jsonEncode` gagal mengubah objek `Address`.
> - Memakai `json['id'] as int` untuk field yang kadang dikirim sebagai String (`"1"`). Jika tidak yakin, gunakan `int.tryParse(json['id'].toString())`.
> - Nama kunci di model tidak sama persis dengan JSON (`userid` vs `userId`). Salin kunci langsung dari contoh response.

4. Persiapan: Package http & Izin Internet Android/iOS

Sebelum request pertama, ada dua hal yang perlu disiapkan: memasang package `http` dan memastikan aplikasi **diizinkan** mengakses internet di setiap platform. Banyak mahasiswa mengira kodenya salah, padahal penyebabnya izin yang belum ditambahkan.

1. Memasang Package `http`

```
flutter pub add http
```

Lalu impor dengan awalan `as http` agar fungsi-fungsinya jelas berasal dari package ini (`http.get`, `http.post`, dan seterusnya):

```dart
import 'package:http/http.dart' as http;

Future<void> cobaKoneksi() async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/posts/1'),
  );
  print('Status: ${response.statusCode}');
  print('Body: ${response.body}');
}
```

2. Android: Izin INTERNET

Mode debug otomatis punya izin internet, tetapi **build release (APK yang dibagikan) tidak**. Tambahkan baris berikut di `android/app/src/main/AndroidManifest.xml`, tepat di atas tag `<application>`:

`android/app/src/main/AndroidManifest.xml`

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:label="aplikasi_saya"
        ...>
```

3. Android: Server Lokal Tanpa HTTPS

Saat backend dijalankan di laptop sendiri (Laravel, Express, dsb.) alamatnya biasanya `http://` tanpa "s". Android 9 ke atas memblokir HTTP biasa (*cleartext*). Untuk keperluan belajar, izinkan lewat atribut di tag `<application>`:

```
<application
    android:usesCleartextTraffic="true"
    ...>
```

> ℹ️ **localhost di Emulator Android**
>
> Di dalam emulator Android, `localhost` berarti emulator itu sendiri, bukan laptop. Gunakan `http://10.0.2.2:8000` untuk mengakses server di laptop. Jika memakai HP fisik, gunakan IP laptop di jaringan Wi-Fi yang sama, misalnya `http://192.168.1.10:8000`.

4. iOS

iOS tidak membutuhkan izin khusus untuk internet. Namun iOS memblokir `http://` (App Transport Security). Untuk server lokal saat belajar, tambahkan di `ios/Runner/Info.plist` di dalam tag `<dict>` utama:

📄 **`ios/Runner/Info.plist`**

```
<key>NSAppTransportSecurity</key>
<dict>
    <key>NSAllowsLocalNetworking</key>
    <true/>
</dict>
```

Untuk server produksi, selalu gunakan `https://` sehingga pengaturan ini tidak diperlukan.

5. macOS Desktop dan Web

- **macOS**: tambahkan `<key>com.apple.security.network.client</key><true/>` di `macos/Runner/DebugProfile.entitlements` dan `Release.entitlements`.
- **Web (Chrome)**: tidak butuh izin, tetapi server harus mengizinkan **CORS**. JSONPlaceholder sudah mengizinkan. Jika API kampus menolak, error yang muncul adalah `ClientException: XMLHttpRequest error`.

Contoh Lengkap: Tes Koneksi

Gunakan aplikasi kecil ini untuk memastikan semua izin sudah benar sebelum lanjut ke section berikutnya.

📄 **`lib/main.dart`**

```dart
// Pastikan sudah: flutter pub add http
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: TesKoneksiScreen()));

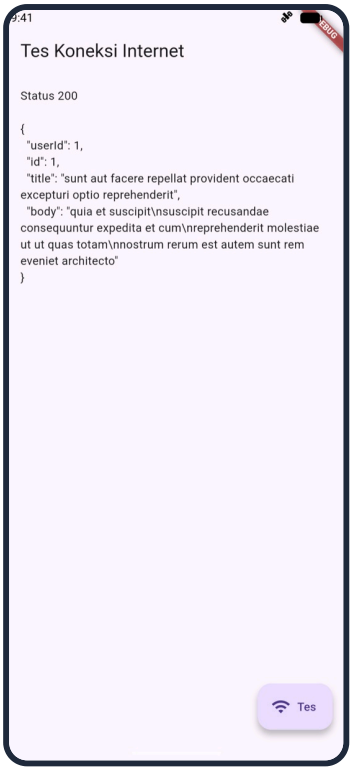
class TesKoneksiScreen extends StatefulWidget {
  const TesKoneksiScreen({super.key});

  @override
  State<TesKoneksiScreen> createState() => _TesKoneksiScreenState();
}
```

```dart
class _TesKoneksiScreenState extends State<TesKoneksiScreen> {
  String _hasil = 'Tekan tombol untuk mengetes koneksi.';
  bool _loading = false;

  Future<void> _tes() async {
    setState(() => _loading = true);
    try {
      final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/posts/1'),
      );
      _hasil = 'Status ${response.statusCode}\n\n${response.body}';
    } catch (e) {
      _hasil = 'Gagal terhubung: $e';
    }
    if (!mounted) return;
    setState(() => _loading = false);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tes Koneksi Internet')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: _loading
            ? const Center(child: CircularProgressIndicator())
            : SingleChildScrollView(child: Text(_hasil)),
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _loading ? null : _tes,
        icon: const Icon(Icons.wifi),
        label: const Text('Tes'),
      ),
    );
  }
}
```



Setelah tombol Tes ditekan: status 200 dan JSON post #1

Jika muncul `Status 200` beserta JSON, koneksi sudah siap. Jika muncul `Failed host lookup`, cek internet emulator atau izin di atas.

> ⚠️ **Kesalahan Umum**
>
> - Aplikasi jalan saat debug tetapi APK release tidak bisa memuat data: lupa izin INTERNET.
> - Memakai `http://localhost:8000` di emulator Android: gunakan `10.0.2.2`.
> - Menambah izin di `android/app/src/debug/AndroidManifest.xml`, padahal yang dipakai release adalah folder `main`.
> - Mengira hot reload sudah cukup setelah mengubah file native (Manifest/Info.plist). Aplikasi harus **di-stop lalu dijalankan ulang**.

5. GET: Mengambil Daftar dan Detail Data

`GET` adalah request yang paling sering dipakai: mengambil daftar data (list) dan mengambil satu data berdasarkan id (detail). Di section ini kita membaca data post dari JSONPlaceholder lalu menampilkannya di layar.

Langkah-langkah Satu Request GET

1. Susun `Uri`.
2. Panggil `await http.get(uri)` untuk mendapatkan `http.Response`.
3. Cek `response.statusCode`.
4. Decode `response.body` dengan `jsonDecode`.
5. Ubah Map/List menjadi model dengan `fromJson`.

Contoh Kecil: GET Daftar

```dart
import 'dart:convert';

import 'package:http/http.dart' as http;

import 'models/post.dart';

Future<List<Post>> fetchPosts() async {
  final uri = Uri.parse('https://jsonplaceholder.typicode.com/posts'); // 1
  final response = await http.get(uri);                                // 2

  if (response.statusCode != 200) {                                    // 3
    throw Exception('Gagal memuat post (HTTP ${response.statusCode})');
  }

  final List<dynamic> data = jsonDecode(response.body) as List<dynamic>; // 4
  return data
      .map((e) => Post.fromJson(e as Map<String, dynamic>))              // 5
      .toList();
}
```

Contoh Kecil: GET Detail dan GET dengan Query

```dart
import 'dart:convert';

import 'package:http/http.dart' as http;

import 'models/post.dart';

// GET /posts/{id} -> satu object
Future<Post> fetchPostById(int id) async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/posts/$id'),
  );
  if (response.statusCode == 404) {
    throw Exception('Post dengan id $id tidak ditemukan');
  }
```

```
  if (response.statusCode != 200) {
    throw Exception('HTTP ${response.statusCode}');
  }
  return Post.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
}

// GET /posts?userId=1 -> filter memakai query parameter
Future<List<Post>> fetchPostsByUser(int userId) async {
  final uri = Uri.https('jsonplaceholder.typicode.com', '/posts', {
    'userId': '$userId',
  });
  final response = await http.get(uri);
  if (response.statusCode != 200) {
    throw Exception('HTTP ${response.statusCode}');
  }
  final data = jsonDecode(response.body) as List<dynamic>;
  return data.map((e) => Post.fromJson(e as Map<String, dynamic>)).toList();
}
```

Perbedaannya hanya pada hasil decode: endpoint daftar mengembalikan **array** (`List`), endpoint detail mengembalikan **object** (`Map`).

Contoh Lengkap: Daftar Post dan Halaman Detail

Kita kumpulkan fungsi GET dalam satu file service, lalu memakainya di UI. Model `Post` diambil dari section sebelumnya (`lib/models/post.dart`).

`lib/services/post_service.dart`

```
import 'dart:convert';

import 'package:http/http.dart' as http;

import '../models/post.dart';

class PostService {
  static const String _baseUrl = 'https://jsonplaceholder.typicode.com';

  Future<List<Post>> fetchPosts({int limit = 20}) async {
    final uri = Uri.parse('$_baseUrl/posts?_limit=$limit');
    final response = await http.get(uri);
    if (response.statusCode != 200) {
      throw Exception('Gagal memuat daftar post (HTTP ${response.statusCode})');
    }
    final data = jsonDecode(response.body) as List<dynamic>;
    return data.map((e) => Post.fromJson(e as Map<String, dynamic>)).toList();
  }

  Future<Post> fetchPost(int id) async {
    final response = await http.get(Uri.parse('$_baseUrl/posts/$id'));
    if (response.statusCode != 200) {
      throw Exception('Gagal memuat post $id (HTTP ${response.statusCode})');
    }
    return Post.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
  }

  // Komentar milik satu post: GET /posts/{id}/comments
```

```
  Future<List<String>> fetchCommentBodies(int postId) async {
    final response = await http.get(Uri.parse('$_baseUrl/posts/$postId/comments'));
    if (response.statusCode != 200) {
      throw Exception('Gagal memuat komentar (HTTP ${response.statusCode})');
    }
    final data = jsonDecode(response.body) as List<dynamic>;
    return data.map((e) => (e as Map<String, dynamic>)['body'] as String).toList();
  }
}
```

lib/main.dart

```
// Pastikan sudah: flutter pub add http
// serta membuat lib/models/post.dart dan lib/services/post_service.dart
import 'package:flutter/material.dart';

import 'models/post.dart';
import 'services/post_service.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const PostListScreen(),
    );
  }
}

class PostListScreen extends StatefulWidget {
  const PostListScreen({super.key});

  @override
  State<PostListScreen> createState() => _PostListScreenState();
}

class _PostListScreenState extends State<PostListScreen> {
  final _service = PostService();
  List<Post> _posts = [];
  bool _loading = true;
  String? _error;

  @override
  void initState() {
    super.initState();
    _load();
  }

  Future<void> _load() async {
    setState(() {
      _loading = true;
      _error = null;
    });
```

```
    try {
      final hasil = await _service.fetchPosts();
      if (!mounted) return;
      setState(() => _posts = hasil);
    } catch (e) {
      if (!mounted) return;
      setState(() => _error = e.toString());
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    Widget body;
    if (_loading) {
      body = const Center(child: CircularProgressIndicator());
    } else if (_error != null) {
      body = Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text(_error!, textAlign: TextAlign.center),
            const SizedBox(height: 12),
            FilledButton(onPressed: _load, child: const Text('Coba Lagi')),
          ],
        ),
      );
    } else {
      body = ListView.separated(
        itemCount: _posts.length,
        separatorBuilder: (context, index) => const Divider(height: 1),
        itemBuilder: (context, index) {
          final post = _posts[index];
          return ListTile(
            leading: CircleAvatar(child: Text('${post.id}')),
            title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis),
            subtitle: Text(post.body, maxLines: 2, overflow: TextOverflow.ellipsis),
            onTap: () => Navigator.push(
              context,
              MaterialPageRoute(builder: (_) => PostDetailScreen(postId: post.id!)),
            ),
          );
        },
      );
    }

    return Scaffold(
      appBar: AppBar(title: const Text('Daftar Post (GET)')),
      body: body,
    );
  }
}

class PostDetailScreen extends StatefulWidget {
  final int postId;
  const PostDetailScreen({super.key, required this.postId});

  @override
```

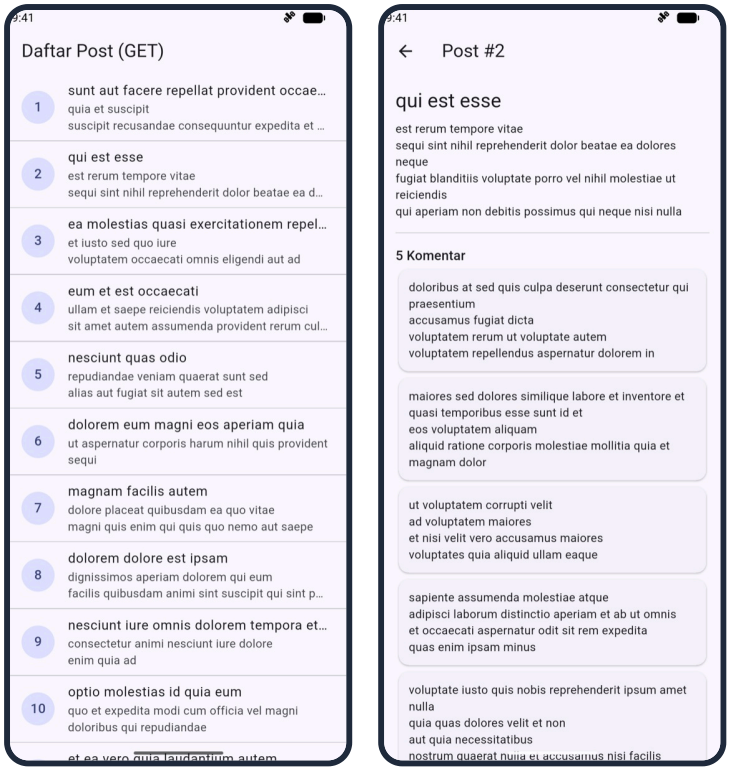
```
  State<PostDetailScreen> createState() => _PostDetailScreenState();
}

class _PostDetailScreenState extends State<PostDetailScreen> {
  final _service = PostService();
  Post? _post;
  List<String> _komentar = [];
  String? _error;

  @override
  void initState() {
    super.initState();
    _load();
  }

  Future<void> _load() async {
    try {
      // Dua request dijalankan bersamaan dengan Future.wait
      final hasil = await Future.wait([
        _service.fetchPost(widget.postId),
        _service.fetchCommentBodies(widget.postId),
      ]);
      if (!mounted) return;
      setState(() {
        _post = hasil[0] as Post;
        _komentar = hasil[1] as List<String>;
      });
    } catch (e) {
      if (!mounted) return;
      setState(() => _error = e.toString());
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Post #${widget.postId}')),
      body: _error != null
          ? Center(child: Text(_error!))
          : _post == null
              ? const Center(child: CircularProgressIndicator())
              : ListView(
                  padding: const EdgeInsets.all(16),
                  children: [
                    Text(_post!.title, style: Theme.of(context).textTheme.headlineSmall),
                    const SizedBox(height: 8),
                    Text(_post!.body),
                    const Divider(height: 32),
                    Text('${_komentar.length} Komentar',
                        style: Theme.of(context).textTheme.titleMedium),
                    for (final k in _komentar)
                      Card(child: Padding(padding: const EdgeInsets.all(12), child: Text(k))),
                  ],
                ),
    );
  }
}
```



Daftar post hasil GET /posts

Halaman detail: GET /posts/2 dan komentarnya

Penjelasan:

- Layar daftar menyimpan tiga state: `_loading`, `_error`, dan `_posts`. Ketiganya menentukan widget apa yang tampil. Di section FutureBuilder nanti, pola ini dibuat lebih ringkas.
- `if (!mounted) return;` setelah `await` mencegah error *setState() called after dispose()* ketika pengguna sudah keluar dari layar sebelum data datang.
- Halaman detail hanya menerima `postId`, lalu mengambil data sendiri. `Future.wait` menjalankan dua request sekaligus sehingga lebih cepat daripada berurutan.
- `?_limit=20` adalah fitur JSONPlaceholder untuk membatasi jumlah data. Setiap API punya nama parameter sendiri (`limit`, `per_page`, `size`), cek dokumentasinya.

Contoh Lain: Memetakan Nama Field yang Berbeda

Kadang nama field di server berbeda dengan model kita. Service berikut mengambil `/users` lalu memetakannya ke `MahasiswaModel` dari section 3 (field `name` menjadi `nama`, dan NIM dibuat dari id sebagai data contoh).

`lib/services/mahasiswa_api_service.dart`

```dart
// lib/services/mahasiswa_api_service.dart
import 'dart:convert';
import 'package:http/http.dart' as http;

import '../models/mahasiswa_model.dart';

class MahasiswaApiService {
  // Ganti dengan endpoint server API kampus Anda atau mock API
```

```dart
  static const String baseUrl = "https://jsonplaceholder.typicode.com";

  // 1. GET: Mengambil daftar mahasiswa
  Future<List<MahasiswaModel>> fetchMahasiswa() async {
    final response = await http.get(
      Uri.parse('$baseUrl/users'),
      headers: {'Accept': 'application/json'},
    ).timeout(const Duration(seconds: 10));

    if (response.statusCode == 200) {
      final List<dynamic> rawList = json.decode(response.body);
      return rawList.map((item) => MahasiswaModel.fromJson({
        'id': item['id'],
        'nama': item['name'],
        'email': item['email'],
        'nim': '2110${item['id']}',
        'avatar': 'https://i.pravatar.cc/150?u=${item['id']}'
      })).toList();
    } else {
      throw Exception("Gagal memuat data dari server (HTTP ${response.statusCode})");
    }
  }

  // 2. POST: Mengirim data mahasiswa baru ke server
  Future<bool> createMahasiswa(MahasiswaModel mhs) async {
    final response = await http.post(
      Uri.parse('$baseUrl/users'),
      headers: {'Content-Type': 'application/json; charset=UTF-8'},
      body: json.encode(mhs.toJson()),
    );

    return response.statusCode == 201; // 201 Created
  }
}
```

⚠️ Kesalahan Umum

- Lupa `await` sehingga variabel berisi `Future<Response>`, bukan `Response`.
- Decode endpoint detail sebagai `List` (atau sebaliknya). Lihat dulu bentuk JSON-nya di browser.
- Memanggil fungsi GET di dalam `build()`. `build()` bisa dipanggil puluhan kali, sehingga request terkirim berulang. Panggil di `initState()`.
- Teks dengan huruf non-ASCII tampil aneh (mis. `Ã©`). Jika server tidak mengirim charset, gunakan `utf8.decode(response.bodyBytes)` sebagai pengganti `response.body`.

6. POST, PUT, PATCH, dan DELETE

Setelah bisa membaca data, sekarang kita **mengirim** data: membuat (POST), mengubah (PUT/PATCH), dan menghapus (DELETE). Ketiga method ini mirip GET, bedanya ada **body JSON** dan **header** `Content-Type`.

> ℹ️ **JSONPlaceholder Hanya Berpura-pura Menyimpan**
>
> JSONPlaceholder menerima POST/PUT/PATCH/DELETE dan membalas seolah berhasil, tetapi **data tidak benar-benar tersimpan**. POST selalu mengembalikan `id: 101`, dan jika kamu GET lagi, data baru tidak muncul. Ini normal. Untuk latihan, kita perbarui list di aplikasi secara lokal berdasarkan response server. Untuk data yang benar-benar tersimpan, gunakan mockapi.io atau backend sendiri (dan Supabase di Modul 12).

Pola Umum Request dengan Body

```dart
import 'dart:convert';

import 'package:http/http.dart' as http;

Future<void> contohPost() async {
  final response = await http.post(
    Uri.parse('https://jsonplaceholder.typicode.com/posts'),
    headers: {'Content-Type': 'application/json; charset=UTF-8'}, // wajib untuk JSON
    body: jsonEncode({'title': 'Halo', 'body': 'Isi', 'userId': 1}), // Map -> String
  );
  print(response.statusCode); // 201
  print(response.body);       // {"title": "Halo", "body": "Isi", "userId": 1, "id": 101}
}
```

PUT vs PATCH

	PUT	PATCH
Isi body	Seluruh field objek	Hanya field yang berubah
Field yang tidak dikirim	Bisa terhapus/di-reset oleh server	Tetap seperti semula
Contoh	`{"id":1,"title":"Baru","body":"...","userId":1}`	`{"title":"Baru"}`
Kapan dipakai	Form edit yang mengirim semua isian	Toggle status, ganti satu field

Contoh Kecil Setiap Method

```dart
import 'dart:convert';

import 'package:http/http.dart' as http;

import 'models/post.dart';

const baseUrl = 'https://jsonplaceholder.typicode.com';
const jsonHeaders = {'Content-Type': 'application/json; charset=UTF-8'};

// CREATE: POST /posts -> 201 Created
Future<Post> createPost(Post post) async {
  final response = await http.post(
    Uri.parse('$baseUrl/posts'),
    headers: jsonHeaders,
    body: jsonEncode(post.toJson()),
  );
  if (response.statusCode != 201) {
    throw Exception('Gagal membuat post (HTTP ${response.statusCode})');
  }
  // Server mengembalikan data yang sudah memiliki id
  return Post.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
}

// UPDATE penuh: PUT /posts/{id} -> 200 OK
Future<Post> updatePost(Post post) async {
  final response = await http.put(
    Uri.parse('$baseUrl/posts/${post.id}'),
    headers: jsonHeaders,
    body: jsonEncode(post.toJson()),
  );
  if (response.statusCode != 200) {
    throw Exception('Gagal mengubah post (HTTP ${response.statusCode})');
  }
  return Post.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
}
```

```dart
// UPDATE sebagian: PATCH /posts/{id} -> 200 OK
Future<Post> updateTitle(int id, String title) async {
  final response = await http.patch(
    Uri.parse('$baseUrl/posts/$id'),
    headers: jsonHeaders,
    body: jsonEncode({'title': title}), // hanya field yang berubah
  );
  if (response.statusCode != 200) {
    throw Exception('Gagal mengubah judul (HTTP ${response.statusCode})');
  }
  return Post.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
}

// DELETE /posts/{id} -> 200 OK (beberapa API memakai 204 No Content)
Future<void> deletePost(int id) async {
  final response = await http.delete(Uri.parse('$baseUrl/posts/$id'));
  if (response.statusCode != 200 && response.statusCode != 204) {
    throw Exception('Gagal menghapus post (HTTP ${response.statusCode})');
  }
}
```

Penjelasan: setiap fungsi mengecek status code yang sesuai standar (POST = 201, lainnya = 200). Fungsi create/update mengembalikan `Post` dari response server karena server bisa menambahkan data (misalnya `id` atau waktu dibuat). DELETE tidak perlu mengembalikan apa pun.

Contoh Lengkap: Playground CRUD

Aplikasi berikut punya 5 tombol (GET, POST, PUT, PATCH, DELETE) dan menampilkan status code serta body response. Cara cepat untuk melihat perilaku setiap method.

lib/main.dart

```dart
// Pastikan sudah: flutter pub add http
import 'dart:convert';

import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: CrudPlaygroundScreen()));

class CrudPlaygroundScreen extends StatefulWidget {
  const CrudPlaygroundScreen({super.key});

  @override
  State<CrudPlaygroundScreen> createState() => _CrudPlaygroundScreenState();
}

class _CrudPlaygroundScreenState extends State<CrudPlaygroundScreen> {
  static const _base = 'https://jsonplaceholder.typicode.com/posts';
  static const _headers = {'Content-Type': 'application/json; charset=UTF-8'};

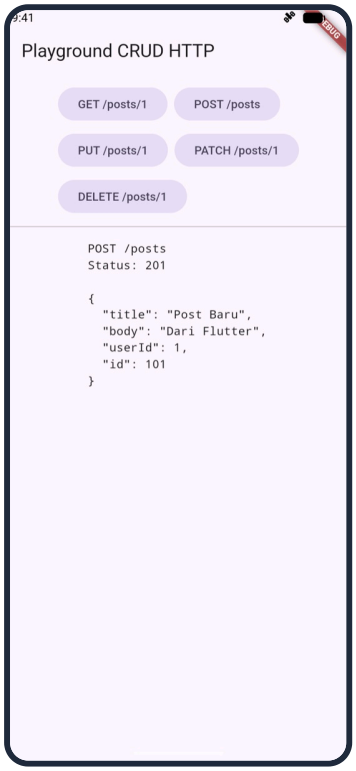
  String _output = 'Pilih salah satu method di bawah.';
  bool _loading = false;
```

```dart
  // Satu fungsi pembantu agar setiap tombol cukup mengirim request-nya
  Future<void> _jalankan(String label, Future<http.Response> Function() request) async {
    setState(() {
      _loading = true;
      _output = 'Mengirim $label ...';
    });
    try {
      final res = await request();
      final pretty = res.body.isEmpty
          ? '(body kosong)'
          : const JsonEncoder.withIndent('  ').convert(jsonDecode(res.body));
      _output = '$label\nStatus: ${res.statusCode}\n\n$pretty';
    } catch (e) {
      _output = '$label gagal: $e';
    }
    if (mounted) setState(() => _loading = false);
  }

  @override
  Widget build(BuildContext context) {
    final tombol = <String, Future<http.Response> Function()>{
      'GET /posts/1': () => http.get(Uri.parse('$_base/1')),
      'POST /posts': () => http.post(
            Uri.parse(_base),
            headers: _headers,
            body: jsonEncode({'title': 'Post Baru', 'body': 'Dari Flutter', 'userId': 1}),
          ),
      'PUT /posts/1': () => http.put(
            Uri.parse('$_base/1'),
            headers: _headers,
            body: jsonEncode({'id': 1, 'title': 'Diganti total', 'body': 'Baru', 'userId': 1}),
          ),
      'PATCH /posts/1': () => http.patch(
            Uri.parse('$_base/1'),
            headers: _headers,
            body: jsonEncode({'title': 'Hanya judul berubah'}),
          ),
      'DELETE /posts/1': () => http.delete(Uri.parse('$_base/1')),
    };

    return Scaffold(
      appBar: AppBar(title: const Text('Playground CRUD HTTP')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12),
            child: Wrap(
              spacing: 8,
              runSpacing: 8,
              children: [
                for (final entry in tombol.entries)
                  FilledButton.tonal(
                    onPressed: _loading ? null : () => _jalankan(entry.key, entry.value),
                    child: Text(entry.key),
                  ),
              ],
            ),
          ),
          const Divider(height: 1),
```

```
          Expanded(
            child: SingleChildScrollView(
              padding: const EdgeInsets.all(16),
              child: SelectableText(
                _output,
                style: const TextStyle(fontFamily: 'monospace'),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```



POST /posts: status 201 dan id 101



PATCH /posts/1: hanya judul yang berubah

Coba tekan **PATCH**: response berisi seluruh post dengan judul baru, sedangkan **PUT** mengembalikan persis yang kita kirim. **DELETE** mengembalikan `{}` dengan status 200.

⚠️ **Kesalahan Umum**

- **Lupa `jsonEncode`**: jika `body` diisi Map langsung, package `http` mengirimnya sebagai form (`title=Halo&body=Isi`), bukan JSON. Server JSON akan menolak atau mengabaikannya.
- **Lupa header `Content-Type`**: banyak server (Laravel, Express) tidak membaca body sebagai JSON tanpa header ini.
- Mengecek `statusCode == 200` untuk POST, padahal standarnya `201` . Lebih aman cek rentang 2xx.
- Menganggap data JSONPlaceholder tersimpan, lalu bingung karena GET tidak menampilkan data baru.
- PUT ke id yang tidak ada (mis. `/posts/101`) di JSONPlaceholder menghasilkan 500. Tangani error ini, jangan biarkan aplikasi crash.

7. Header & Bearer Token

Kebanyakan API sungguhan (SIA kampus, marketplace) hanya memberikan data kepada pengguna yang sudah login. Setelah login, server memberi **token**. Token itu kemudian dikirim di setiap request melalui **header** `Authorization` dengan format `Bearer <token>`.

Alur Autentikasi Token

```
1. POST /login  {email, password}        -->  server cek akun
                                          <--  200 {"token": "abc.def.ghi"}
2. Simpan token (memori / SharedPreferences, lihat Modul 10)
3. GET /profil
   Authorization: Bearer abc.def.ghi      -->  server cek token
                                          <--  200 {data profil}
4. Token salah / kedaluwarsa              <--  401 Unauthorized  -> arahkan ke login
```

Contoh Kecil: Menambahkan Header

Parameter `headers` menerima `Map<String, String>`. Perhatikan ada **spasi** setelah kata `Bearer`.

```dart
import 'package:http/http.dart' as http;

Future<void> ambilProfil(String token) async {
  final response = await http.get(
    Uri.parse('https://httpbin.org/bearer'),
    headers: {
      'Accept': 'application/json',
      'Authorization': 'Bearer $token',
    },
  );
  print(response.statusCode); // 200 jika ada token, 401 jika tidak
}
```

Kita memakai httpbin.org, layanan uji gratis. Endpoint `/bearer` membalas 200 jika header Bearer ada dan 401 jika tidak, sedangkan `/headers` mengembalikan semua header yang kita kirim.

Fungsi Pembuat Header

Daripada menulis header berulang di setiap request, buat satu fungsi:

```dart
Map<String, String> buildHeaders({String? token, bool json = true}) {
  return {
    'Accept': 'application/json',
    if (json) 'Content-Type': 'application/json; charset=UTF-8',
    if (token != null) 'Authorization': 'Bearer $token',
  };
}
```

Cara Lebih Rapi: `http.BaseClient` yang Menyisipkan Token Otomatis

Kita bisa membuat client sendiri yang otomatis menambahkan header ke **setiap** request. Ini mirip "interceptor" di library lain.

`lib/services/auth_client.dart`

```dart
import 'package:http/http.dart' as http;

class AuthClient extends http.BaseClient {
  final http.Client _inner;
  String? token; // bisa diganti setelah login / logout

  AuthClient({http.Client? inner, this.token}) : _inner = inner ?? http.Client();

  @override
  Future<http.StreamedResponse> send(http.BaseRequest request) {
    request.headers['Accept'] = 'application/json';
    final t = token;
    if (t != null) {
      request.headers['Authorization'] = 'Bearer $t';
    }
    return _inner.send(request);
  }

  @override
  void close() {
    _inner.close();
    super.close();
  }
}
```

`AuthClient` bisa dipakai persis seperti package `http`: `client.get(...)`, `client.post(...)`, dan seterusnya. Semua method itu pada akhirnya memanggil `send()`, sehingga header pasti ikut.

Contoh Lengkap: Simulasi Login dengan Token

`lib/main.dart`

```dart
// Pastikan sudah: flutter pub add http, dan membuat lib/services/auth_client.dart
import 'package:flutter/material.dart';

import 'services/auth_client.dart';

void main() => runApp(const MaterialApp(home: TokenDemoScreen()));

class TokenDemoScreen extends StatefulWidget {
  const TokenDemoScreen({super.key});

  @override
  State<TokenDemoScreen> createState() => _TokenDemoScreenState();
}

class _TokenDemoScreenState extends State<TokenDemoScreen> {
  final _client = AuthClient();
  String _hasil = '-';

  @override
  void dispose() {
    _client.close(); // tutup koneksi saat layar dibuang
    super.dispose();
  }
```

```
  void _login() {
    // Pada aplikasi nyata token didapat dari response POST /login
    setState(() => _client.token = 'token-rahasia-mahasiswa-123');
  }

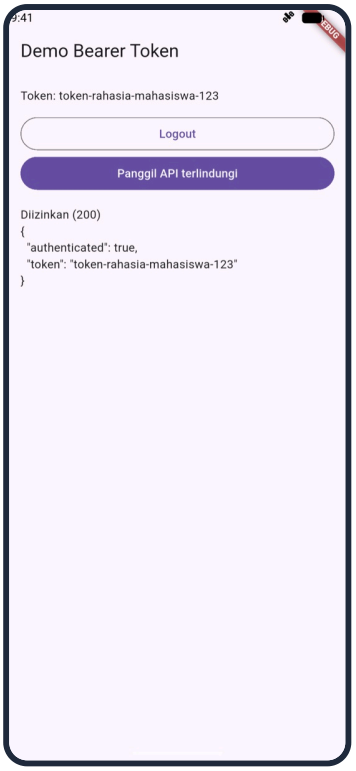
  void _logout() {
    setState(() => _client.token = null);
  }

  Future<void> _panggilApi() async {
    setState(() => _hasil = 'Memanggil /bearer ...');
    try {
      final res = await _client.get(Uri.parse('https://httpbin.org/bearer'));
      final pesan = switch (res.statusCode) {
        200 => 'Diizinkan (200)\n${res.body}',
        401 => 'Ditolak (401 Unauthorized): silakan login dulu',
        _ => 'Status tak terduga: ${res.statusCode}',
      };
      if (mounted) setState(() => _hasil = pesan);
    } catch (e) {
      if (mounted) setState(() => _hasil = 'Error jaringan: $e');
    }
  }

  @override
  Widget build(BuildContext context) {
    final sudahLogin = _client.token != null;
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Bearer Token')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text(sudahLogin ? 'Token: ${_client.token}' : 'Belum login (tanpa token)'),
            const SizedBox(height: 12),
            OutlinedButton(
              onPressed: sudahLogin ? _logout : _login,
              child: Text(sudahLogin ? 'Logout' : 'Login (dapat token)'),
            ),
            FilledButton(onPressed: _panggilApi, child: const Text('Panggil API terlindungi')),
            const SizedBox(height: 16),
            Expanded(child: SingleChildScrollView(child: Text(_hasil))),
          ],
        ),
      ),
    );
  }
}
```



Memanggil API sebelum login: 401



Setelah login: token terkirim, status 200

Penjelasan: panggil API sebelum login, hasilnya 401. Tekan Login lalu panggil lagi, hasilnya 200 dan httpbin menampilkan token yang diterimanya. Ekspresi `switch` (Dart 3) membuat penanganan setiap status code ringkas.

⚠️ Kesalahan Umum & Keamanan

- Menulis `'Bearer$token'` tanpa spasi, atau `'bearer'` huruf kecil pada server yang ketat.
- Menyimpan **API key rahasia** (kunci admin, secret key) di kode Flutter. Aplikasi bisa dibongkar; kunci rahasia harus tetap di server.
- Token login yang sensitif sebaiknya disimpan dengan `flutter_secure_storage`. SharedPreferences (Modul 10) cukup untuk latihan, tetapi tidak terenkripsi.
- Tidak menangani 401. Saat token kedaluwarsa, hapus token dan arahkan pengguna ke halaman login.

8. Penanganan Error, Timeout, dan Offline

Jaringan itu tidak bisa diandalkan: sinyal hilang di dalam lift, server kampus down saat KRS-an, atau URL salah ketik. Aplikasi yang baik **tidak crash** dan memberi pesan yang dimengerti pengguna. Di section ini kita mengenali jenis-jenis kegagalan dan cara menanganinya satu per satu.

Dua Jenis Kegagalan

Jenis	Contoh	Bagaimana terdeteksi
Request sampai, tetapi server menolak	404, 401, 500	Tidak ada exception! Harus cek `response.statusCode`.
Tidak ada internet / host salah	Mode pesawat, DNS gagal	`SocketException` / `http.ClientException`
Server terlalu lama	Server lambat, sinyal lemah	`TimeoutException` (jika memakai `.timeout()`)
Response bukan JSON yang diharapkan	Halaman HTML error	`FormatException` / `TypeError`

Langkah 1: Cek Status Code

```dart
import 'package:http/http.dart' as http;

Future<String> ambil() async {
  final response = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts/9999'));

  // http TIDAK melempar error untuk 404, jadi kita cek sendiri
  if (response.statusCode >= 200 && response.statusCode < 300) {
    return response.body;
  }
  throw Exception('Server membalas ${response.statusCode}');
}
```

Langkah 2: Tangkap Exception dengan `try/catch` dan `on`

Klausa `on NamaException catch (e)` menangkap jenis error tertentu. Urutannya dari yang paling spesifik ke paling umum.

```dart
import 'dart:async'; // TimeoutException
import 'dart:io'; // SocketException (tidak tersedia di web)

import 'package:http/http.dart' as http;
```

```dart
Future<String> ambilDenganPenanganan() async {
  try {
    final response = await http
        .get(Uri.parse('https://jsonplaceholder.typicode.com/posts/1'))
        .timeout(const Duration(seconds: 10)); // batas waktu 10 detik
    return 'Status ${response.statusCode}';
  } on SocketException {
    return 'Tidak ada koneksi internet.';
  } on TimeoutException {
    return 'Server terlalu lama merespons. Coba lagi.';
  } on http.ClientException catch (e) {
    return 'Gagal menghubungi server: ${e.message}';
  } catch (e) {
    return 'Terjadi kesalahan tak terduga: $e';
  }
}
```

> ℹ️ **SocketException vs ClientException**
>
> Di Android/iOS, error koneksi dari package `http` berbentuk `ClientException` yang *juga* merupakan `SocketException`, sehingga kedua klausa di atas bisa menangkapnya. Di Flutter Web, `dart:io` tidak berfungsi dan error koneksi hanya berupa `http.ClientException`. Karena itu selalu sediakan klausa `http.ClientException`.

Langkah 3: Exception Buatan Sendiri

Agar UI cukup menampilkan satu pesan yang ramah, ubah semua jenis error menjadi satu class `ApiException`:

📄 **`lib/services/api_exception.dart`**

```dart
class ApiException implements Exception {
  final String message;
  final int? statusCode;

  const ApiException(this.message, {this.statusCode});

  // Menerjemahkan status code menjadi pesan yang dimengerti pengguna
  factory ApiException.fromStatus(int code) {
    final pesan = switch (code) {
      400 => 'Data yang dikirim tidak valid.',
      401 => 'Sesi berakhir, silakan login ulang.',
      403 => 'Anda tidak punya akses ke data ini.',
      404 => 'Data tidak ditemukan.',
      422 => 'Validasi gagal, periksa kembali isian.',
      >= 500 => 'Server sedang bermasalah. Coba beberapa saat lagi.',
      _ => 'Terjadi kesalahan (kode $code).',
    };
    return ApiException(pesan, statusCode: code);
  }
```

```
  @override
  String toString() => message; // agar Text('$error') langsung rapi
}
```

Contoh Lengkap: Simulator Error

Aplikasi ini punya tombol untuk memicu setiap jenis kegagalan: sukses, 404, host tidak ada, dan timeout (endpoint httpbin `/delay/5` butuh 5 detik, sedangkan batas kita 2 detik). Coba juga nyalakan mode pesawat di emulator.

lib/main.dart

```dart
// Pastikan sudah: flutter pub add http, dan membuat lib/services/api_exception.dart
import 'dart:async';
import 'dart:convert';
import 'dart:io';

import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

import 'services/api_exception.dart';

/// Satu fungsi GET yang mengubah semua jenis kegagalan menjadi ApiException.
Future<dynamic> getJson(String url, {Duration timeout = const Duration(seconds: 10)}) async {
  try {
    final res = await http.get(Uri.parse(url)).timeout(timeout);
    if (res.statusCode < 200 || res.statusCode >= 300) {
      throw ApiException.fromStatus(res.statusCode);
    }
    return jsonDecode(res.body);
  } on SocketException {
    throw const ApiException('Tidak ada koneksi internet. Periksa Wi-Fi atau data seluler.');
  } on TimeoutException {
    throw const ApiException('Koneksi lambat, server tidak merespons tepat waktu.');
  } on http.ClientException {
    throw const ApiException('Gagal menghubungi server.');
  } on FormatException {
    throw const ApiException('Format data dari server tidak dikenali.');
  }
}

void main() => runApp(const MaterialApp(home: ErrorDemoScreen()));

class ErrorDemoScreen extends StatefulWidget {
  const ErrorDemoScreen({super.key});

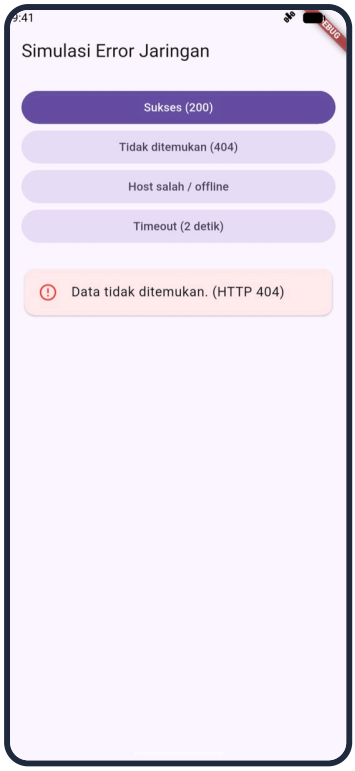
  @override
  State<ErrorDemoScreen> createState() => _ErrorDemoScreenState();
}

class _ErrorDemoScreenState extends State<ErrorDemoScreen> {
  String _pesan = 'Pilih skenario.';
  bool _sukses = true;
  bool _loading = false;

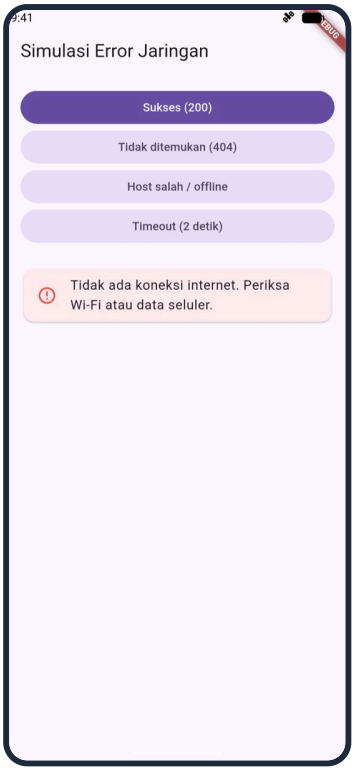
  Future<void> _coba(String url, {Duration timeout = const Duration(seconds: 10)}) async {
    setState(() => _loading = true);
```

```
    try {
      final data = await getJson(url, timeout: timeout);
      _sukses = true;
      _pesan = 'Berhasil: ${(data as Map)['title']}';
    } on ApiException catch (e) {
      _sukses = false;
      _pesan = e.statusCode == null ? e.message : '${e.message} (HTTP ${e.statusCode})';
    }
    if (mounted) setState(() => _loading = false);
  }

  @override
  Widget build(BuildContext context) {
    const base = 'https://jsonplaceholder.typicode.com';
    return Scaffold(
      appBar: AppBar(title: const Text('Simulasi Error Jaringan')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          FilledButton(onPressed: () => _coba('$base/posts/1'), child: const Text('Sukses (200)')),
          FilledButton.tonal(onPressed: () => _coba('$base/posts/9999'), child: const Text('Tidak
ditemukan (404)')),
          FilledButton.tonal(
            onPressed: () => _coba('https://host-tidak-ada.invalid/data'),
            child: const Text('Host salah / offline'),
          ),
          FilledButton.tonal(
            onPressed: () => _coba('https://httpbin.org/delay/5', timeout: const Duration(seconds:
2)),
            child: const Text('Timeout (2 detik)'),
          ),
          const SizedBox(height: 24),
          if (_loading)
            const Center(child: CircularProgressIndicator())
          else
            Card(
              color: _sukses ? Colors.green.shade50 : Colors.red.shade50,
              child: ListTile(
                leading: Icon(
                  _sukses ? Icons.check_circle : Icons.error_outline,
                  color: _sukses ? Colors.green : Colors.red,
                ),
                title: Text(_pesan),
              ),
            ),
        ],
      ),
    );
  }
}
```



Skenario 404 diterjemahkan menjadi pesan ramah



Host tidak ditemukan / offline

Penjelasan: fungsi `getJson` adalah satu-satunya tempat yang tahu soal `SocketException`, `TimeoutException`, dan status code. UI hanya perlu menangkap `ApiException` dan menampilkan `message`. Pola ini akan kita pakai lagi di service/repository layer.

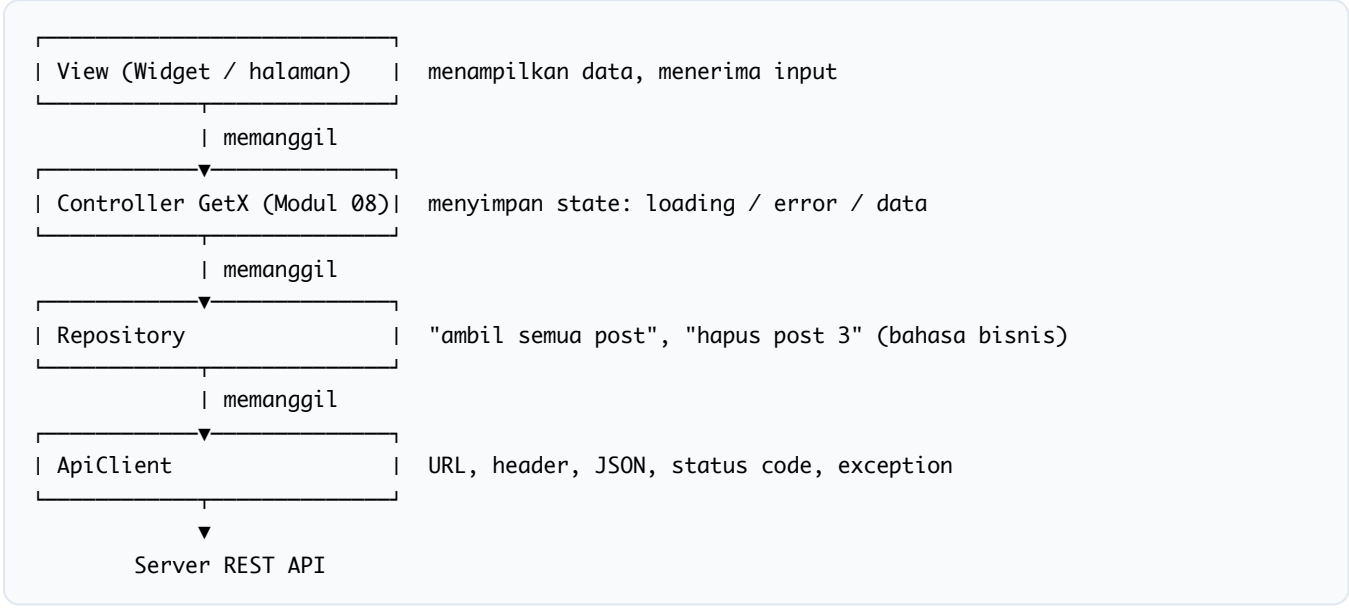
⚠️ Kesalahan Umum

- Hanya memakai `try/catch` tanpa mengecek status code, sehingga response 500 dianggap sukses lalu `jsonDecode` gagal dengan pesan membingungkan.
- Menampilkan `e.toString()` mentah kepada pengguna (`ClientException with SocketException: Failed host lookup...`). Terjemahkan menjadi kalimat yang ramah.
- Tidak memberi `.timeout()`: tanpa batas waktu, spinner bisa berputar sangat lama.
- Menelan error diam-diam: `catch (e) {}` tanpa pesan apa pun membuat pengguna dan programmer sama-sama bingung.

9. Service Layer: ApiClient & Repository

Jika kode `http.get`, `jsonDecode`, dan pengecekan error ditulis langsung di dalam widget, file UI cepat membengkak dan kode yang sama tersebar di mana-mana. Solusinya adalah membagi kode menjadi **lapisan (layer)** yang masing-masing punya satu tugas.

Pembagian Lapisan



Jika dicocokkan dengan pola MVC di Modul 08: **Repository** dan **ApiClient** berperan sebagai lapisan *service* (akses data), `Post` adalah *model*, lalu controller dan view ditambahkan di section 12.

Lapisan	Tahu tentang	Tidak boleh tahu tentang
ApiClient	HTTP, header, JSON mentah, status code	Model `Post`, widget
Repository	Endpoint, model, `fromJson` / `toJson`	Widget, `BuildContext`
View / Controller	Model dan method repository	`http`, `jsonDecode`, URL

1. ApiClient: Satu Pintu untuk Semua Request

File ini memakai `ApiException` dari section sebelumnya (salin ke `lib/core/api_exception.dart`).

`lib/core/api_exception.dart`

```dart
class ApiException implements Exception {
  final String message;
  final int? statusCode;

  const ApiException(this.message, {this.statusCode});
```

```
  factory ApiException.fromStatus(int code) {
    final pesan = switch (code) {
      400 => 'Data yang dikirim tidak valid.',
      401 => 'Sesi berakhir, silakan login ulang.',
      403 => 'Anda tidak punya akses ke data ini.',
      404 => 'Data tidak ditemukan.',
      422 => 'Validasi gagal, periksa kembali isian.',
      >= 500 => 'Server sedang bermasalah. Coba beberapa saat lagi.',
      _ => 'Terjadi kesalahan (kode $code).',
    };
    return ApiException(pesan, statusCode: code);
  }

  @override
  String toString() => message;
}
```

lib/core/api_client.dart

```
import 'dart:async';
import 'dart:convert';
import 'dart:io';

import 'package:http/http.dart' as http;

import 'api_exception.dart';

class ApiClient {
  final String baseUrl;
  final http.Client _http;
  final Duration timeout;
  String? token; // diisi setelah login (opsional)

  ApiClient({
    required this.baseUrl,
    http.Client? httpClient,
    this.timeout = const Duration(seconds: 15),
  }) : _http = httpClient ?? http.Client();

  Map<String, String> get _headers => {
        'Accept': 'application/json',
        'Content-Type': 'application/json; charset=UTF-8',
        if (token != null) 'Authorization': 'Bearer $token',
      };

  Uri _uri(String path, [Map<String, String>? query]) =>
      Uri.parse('$baseUrl$path').replace(queryParameters: query);

  Future<dynamic> get(String path, {Map<String, String>? query}) =>
      _send(() => _http.get(_uri(path, query), headers: _headers));

  Future<dynamic> post(String path, Object body) =>
      _send(() => _http.post(_uri(path), headers: _headers, body: jsonEncode(body)));

  Future<dynamic> put(String path, Object body) =>
      _send(() => _http.put(_uri(path), headers: _headers, body: jsonEncode(body)));
```

```
  Future<dynamic> patch(String path, Object body) =>
      _send(() => _http.patch(_uri(path), headers: _headers, body: jsonEncode(body)));

  Future<dynamic> delete(String path) =>
      _send(() => _http.delete(_uri(path), headers: _headers));

  // Semua request lewat sini: timeout, cek status, decode, terjemahkan error
  Future<dynamic> _send(Future<http.Response> Function() request) async {
    try {
      final res = await request().timeout(timeout);
      if (res.statusCode < 200 || res.statusCode >= 300) {
        throw ApiException.fromStatus(res.statusCode);
      }
      if (res.body.isEmpty) return null; // mis. 204 No Content
      return jsonDecode(utf8.decode(res.bodyBytes));
    } on SocketException {
      throw const ApiException('Tidak ada koneksi internet.');
    } on TimeoutException {
      throw const ApiException('Server tidak merespons. Coba lagi.');
    } on http.ClientException {
      throw const ApiException('Gagal menghubungi server.');
    } on FormatException {
      throw const ApiException('Format data dari server tidak valid.');
    }
  }
}
```

Penjelasan: lima method publik hanya berbeda pada fungsi `http` yang dipanggil. Semua logika berat ada di `_send`, sehingga cukup ditulis sekali. `Uri.replace(queryParameters: null)` tidak menambah apa pun jika query kosong.

2. Repository: Bahasa Bisnis

Repository dibuat sebagai `abstract class` (kontrak) plus implementasinya. Keuntungannya: UI bisa diuji dengan repository palsu (*fake*) tanpa internet, dan backend bisa diganti (mis. ke Supabase) tanpa mengubah UI.

`lib/repositories/post_repository.dart`

```
import '../core/api_client.dart';
import '../models/post.dart';

/// Kontrak: apa saja yang bisa dilakukan terhadap data Post.
abstract class PostRepository {
  Future<List<Post>> getAll();
  Future<Post> getById(int id);
  Future<Post> create(Post post);
  Future<Post> update(Post post);
  Future<void> delete(int id);
}

/// Implementasi yang memakai REST API JSONPlaceholder.
class RemotePostRepository implements PostRepository {
  final ApiClient _api;
  RemotePostRepository(this._api);
```

```dart
  @override
  Future<List<Post>> getAll() async {
    final data = await _api.get('/posts', query: {'_limit': '20'}) as List<dynamic>;
    return data.map((e) => Post.fromJson(e as Map<String, dynamic>)).toList();
  }

  @override
  Future<Post> getById(int id) async {
    final data = await _api.get('/posts/$id') as Map<String, dynamic>;
    return Post.fromJson(data);
  }

  @override
  Future<Post> create(Post post) async {
    final data = await _api.post('/posts', post.toJson()) as Map<String, dynamic>;
    return Post.fromJson(data);
  }

  @override
  Future<Post> update(Post post) async {
    final data = await _api.put('/posts/${post.id}', post.toJson()) as Map<String, dynamic>;
    return Post.fromJson(data);
  }

  @override
  Future<void> delete(int id) => _api.delete('/posts/$id');
}

/// Implementasi palsu di memori: untuk testing atau saat server belum siap.
class FakePostRepository implements PostRepository {
  final List<Post> _data = [
    const Post(id: 1, userId: 1, title: 'Post palsu pertama', body: 'Tanpa internet'),
    const Post(id: 2, userId: 1, title: 'Post palsu kedua', body: 'Cocok untuk testing'),
  ];

  @override
  Future<List<Post>> getAll() async {
    await Future.delayed(const Duration(milliseconds: 500)); // simulasi jaringan
    return List.of(_data);
  }

  @override
  Future<Post> getById(int id) async => _data.firstWhere((p) => p.id == id);

  @override
  Future<Post> create(Post post) async {
    final baru = post.copyWith(id: _data.length + 1);
    _data.add(baru);
    return baru;
  }

  @override
  Future<Post> update(Post post) async {
    final i = _data.indexWhere((p) => p.id == post.id);
    _data[i] = post;
    return post;
  }

  @override
```

```
  Future<void> delete(int id) async => _data.removeWhere((p) => p.id == id);
}
```

3. Memakai Repository di UI

Perhatikan: file UI berikut **tidak** mengimpor `http` maupun `dart:convert`. Ubah variabel `pakaiDataPalsu` menjadi `true` untuk mencoba tanpa internet.

`lib/main.dart`

```
// Butuh: lib/models/post.dart, lib/core/api_exception.dart,
// lib/core/api_client.dart, lib/repositories/post_repository.dart
import 'package:flutter/material.dart';

import 'core/api_client.dart';
import 'models/post.dart';
import 'repositories/post_repository.dart';

const pakaiDataPalsu = false;

void main() {
  final PostRepository repo = pakaiDataPalsu
      ? FakePostRepository()
      : RemotePostRepository(ApiClient(baseUrl: 'https://jsonplaceholder.typicode.com'));
  runApp(MaterialApp(home: PostScreen(repository: repo)));
}

class PostScreen extends StatefulWidget {
  final PostRepository repository;
  const PostScreen({super.key, required this.repository});

  @override
  State<PostScreen> createState() => _PostScreenState();
}

class _PostScreenState extends State<PostScreen> {
  List<Post> _posts = [];
  String? _error;
  bool _loading = true;

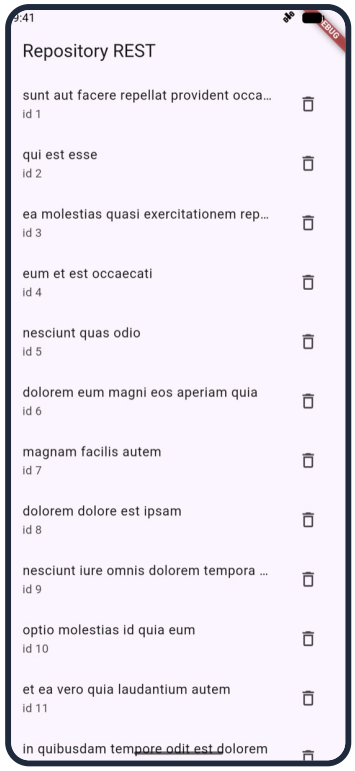
  @override
  void initState() {
    super.initState();
    _load();
  }

  Future<void> _load() async {
    try {
      final posts = await widget.repository.getAll();
      if (mounted) setState(() => _posts = posts);
    } catch (e) {
      if (mounted) setState(() => _error = '$e');
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  Future<void> _hapus(Post post) async {
```

```
    try {
      await widget.repository.delete(post.id!);
      setState(() => _posts.remove(post));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('$e')));
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(pakaiDataPalsu ? 'Repository Palsu' : 'Repository REST')),
      body: _loading
          ? const Center(child: CircularProgressIndicator())
          : _error != null
              ? Center(child: Text(_error!))
              : ListView(
                  children: [
                    for (final post in _posts)
                      ListTile(
                        title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis),
                        subtitle: Text('id ${post.id}'),
                        trailing: IconButton(
                          icon: const Icon(Icons.delete_outline),
                          onPressed: () => _hapus(post),
                        ),
                      ),
                  ],
                ),
    );
  }
}
```



UI memakai RemotePostRepository tanpa kode HTTP

💡 **Kenapa Repot Membuat Lapisan?**

- **Mudah diganti**: pindah dari REST ke Supabase (Modul 12) cukup membuat `SupabasePostRepository` baru.
- **Mudah diuji**: Modul 15 (Testing) memakai `FakePostRepository` agar tes tidak bergantung internet.
- **Mudah dibaca**: `repository.delete(3)` jauh lebih jelas daripada 10 baris kode HTTP.

⚠️ **Kesalahan Umum**

- Mengimpor `package:flutter/material.dart` di repository atau ApiClient. Lapisan data tidak boleh bergantung pada UI.
- Membuat `http.Client()` baru di setiap method tanpa ditutup. Buat sekali dan gunakan ulang.
- Menangkap error di repository lalu mengembalikan list kosong. UI jadi tidak tahu bahwa terjadi error; biarkan exception naik ke controller/UI.

10. Menangani 3 Kondisi UI dengan FutureBuilder

Di section GET kita menulis tiga variabel state (`_loading`, `_error`, `_posts`) secara manual. Widget **`FutureBuilder`** melakukan pekerjaan itu untuk kita: ia mengamati sebuah `Future` dan otomatis membangun ulang UI sesuai statusnya.

Konsep: `AsyncSnapshot`

Fungsi `builder` menerima `snapshot` yang berisi keadaan Future saat ini:

Properti	Arti	Tampilkan
`snapshot.connectionState == ConnectionState.waiting`	Future sedang berjalan	Loading (spinner)
`snapshot.hasError` / `snapshot.error`	Future selesai dengan exception	Pesan error + tombol coba lagi
`snapshot.hasData` / `snapshot.data`	Future selesai dengan nilai	Data (list, detail)
`ConnectionState.done`	Future sudah selesai (sukses atau error)	-

Contoh Kecil

```dart
import 'package:flutter/material.dart';

class JamServer extends StatefulWidget {
  const JamServer({super.key});

  @override
  State<JamServer> createState() => _JamServerState();
}

class _JamServerState extends State<JamServer> {
  // Future disimpan di field, dibuat sekali di initState
  late final Future<String> _future;

  @override
  void initState() {
    super.initState();
    _future = Future.delayed(const Duration(seconds: 2), () => 'Data siap!');
  }

  @override
  Widget build(BuildContext context) {
    return FutureBuilder<String>(
      future: _future,
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
```

```
          return const CircularProgressIndicator(); // 1. loading
        }
        if (snapshot.hasError) {
          return Text('Error: ${snapshot.error}'); // 2. error
        }
        return Text(snapshot.data ?? '-'); // 3. data
      },
    );
  }
}
```

Urutan pengecekan selalu sama: **loading → error → data**. Jika urutannya terbalik, layar bisa menampilkan data lama ketika sedang memuat ulang.

Contoh Lengkap: Daftar Mahasiswa dari API

Contoh ini memakai `MahasiswaModel` (section 3) dan `MahasiswaApiService` (section 5). Tombol *Coba Lagi* bekerja dengan **mengganti Future** di dalam `setState`, sehingga FutureBuilder kembali ke status waiting lalu memuat ulang.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: flutter pub add http, serta membuat file model & service di atas.
import 'package:flutter/material.dart';

import 'models/mahasiswa_model.dart';
import 'services/mahasiswa_api_service.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: DaftarMahasiswaApiScreen(),
    );
  }
}

class DaftarMahasiswaApiScreen extends StatefulWidget {
  const DaftarMahasiswaApiScreen({super.key});

  @override
  State<DaftarMahasiswaApiScreen> createState() => _DaftarMahasiswaApiScreenState();
}

class _DaftarMahasiswaApiScreenState extends State<DaftarMahasiswaApiScreen> {
  late Future<List<MahasiswaModel>> _futureData;
  final MahasiswaApiService _apiService = MahasiswaApiService();
```

```dart
  @override
  void initState() {
    super.initState();
    // Simpan Future di initState agar tidak terpanggil berulang saat rebuild!
    _futureData = _apiService.fetchMahasiswa();
  }

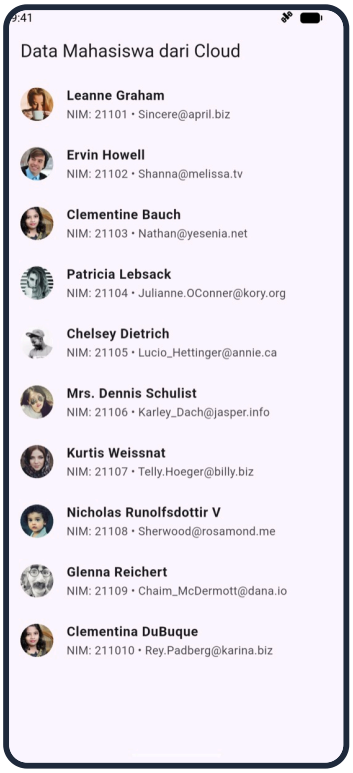
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Data Mahasiswa dari Cloud")),
      body: FutureBuilder<List<MahasiswaModel>>(
        future: _futureData,
        builder: (context, snapshot) {
          // KONDISI 1: Sedang Menunggu Respon Jaringan (Loading)
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  CircularProgressIndicator(),
                  SizedBox(height: 12),
                  Text("Mengunduh data dari server kampus..."),
                ],
              ),
            );
          }

          // KONDISI 2: Terjadi Kesalahan Jaringan / Server Error
          if (snapshot.hasError) {
            return Center(
              child: Padding(
                padding: const EdgeInsets.all(20),
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    const Icon(Icons.cloud_off, size: 64, color: Colors.red),
                    const SizedBox(height: 12),
                    Text("Terjadi Error: ${snapshot.error}"),
                    const SizedBox(height: 16),
                    ElevatedButton(
                      onPressed: () => setState(() => _futureData = _apiService.fetchMahasiswa()),
                      child: const Text("Coba Lagi"),
                    ),
                  ],
                ),
              ),
            );
          }

          // KONDISI 3: Data Berhasil Diterima
          final listMhs = snapshot.data ?? [];
          if (listMhs.isEmpty) {
            return const Center(child: Text("Tidak ada data mahasiswa ditemukan."));
          }

          return ListView.builder(
            itemCount: listMhs.length,
            itemBuilder: (context, index) {
```

```
              final mhs = listMhs[index];
              return ListTile(
                leading: CircleAvatar(
                  // foregroundImage + child: jika foto gagal dimuat, huruf awal nama tetap tampil
                  foregroundImage: NetworkImage(mhs.avatarUrl),
                  onForegroundImageError: (error, stackTrace) {},
                  child: Text(mhs.nama.isNotEmpty ? mhs.nama[0] : '?'),
                ),
                title: Text(mhs.nama, style: const TextStyle(fontWeight: FontWeight.bold)),
                subtitle: Text("NIM: ${mhs.nim} • ${mhs.email}"),
              );
            },
          );
        },
      ),
    );
  }
}
```



Kondisi 3 FutureBuilder: data berhasil dimuat

Penjelasan:

- `late Future<...> _futureData` diisi di `initState`. Jika fungsi API dipanggil langsung di parameter `future:`, setiap rebuild (mis. keyboard muncul) akan mengirim request baru.
- Kondisi 3 juga menangani list kosong. Data kosong bukan error, tetapi tetap perlu pesan agar layar tidak tampak "rusak".
- `foregroundImage` + `child` pada `CircleAvatar` membuat huruf awal tetap tampil jika foto gagal dimuat.

⚠️ **Kesalahan Umum**

- `future: _apiService.fetchMahasiswa()` langsung di `build()`: request berulang tanpa henti.
- Hanya memeriksa `snapshot.hasData` tanpa `hasError`, sehingga saat error spinner berputar selamanya.
- Memakai `snapshot.data!` sebelum memastikan statusnya, menyebabkan *Null check operator used on a null value*.
- Data tidak muncul sama sekali di APK release: cek izin INTERNET (section 4).

11. Pull-to-Refresh dengan RefreshIndicator

Pengguna terbiasa **menarik layar ke bawah** untuk memuat ulang data (Instagram, Gmail). Di Flutter, gestur ini disediakan widget `RefreshIndicator`.

Konsep

- `RefreshIndicator` membungkus widget yang bisa di-scroll (`ListView`, `CustomScrollView`).
- Parameter `onRefresh` wajib mengembalikan `Future`. Spinner tetap tampil sampai Future itu selesai.
- Child harus bisa di-scroll **meskipun isinya sedikit**. Gunakan `physics: const AlwaysScrollableScrollPhysics()`.

Contoh Kecil

```dart
import 'package:flutter/material.dart';

class AngkaAcak extends StatefulWidget {
  const AngkaAcak({super.key});

  @override
  State<AngkaAcak> createState() => _AngkaAcakState();
}

class _AngkaAcakState extends State<AngkaAcak> {
  int _versi = 1;

  Future<void> _refresh() async {
    await Future.delayed(const Duration(seconds: 1)); // pura-pura memanggil API
    setState(() => _versi++);
  }

  @override
  Widget build(BuildContext context) {
    return RefreshIndicator(
      onRefresh: _refresh,
      child: ListView(
        physics: const AlwaysScrollableScrollPhysics(),
        children: [ListTile(title: Text('Data versi $_versi'))],
      ),
    );
  }
}
```

Contoh Lengkap: FutureBuilder + Pull-to-Refresh

Memakai `Post` (section 3) dan `PostService` (section 5). Kuncinya: `_refresh()` membuat Future baru, menyimpannya lewat `setState`, lalu **menunggu** Future yang sama agar spinner RefreshIndicator berhenti tepat ketika data tiba.

lib/main.dart

```dart
// Butuh: flutter pub add http, lib/models/post.dart, lib/services/post_service.dart
import 'package:flutter/material.dart';

import 'models/post.dart';
import 'services/post_service.dart';

void main() => runApp(const MaterialApp(home: RefreshPostScreen()));

class RefreshPostScreen extends StatefulWidget {
  const RefreshPostScreen({super.key});

  @override
  State<RefreshPostScreen> createState() => _RefreshPostScreenState();
}

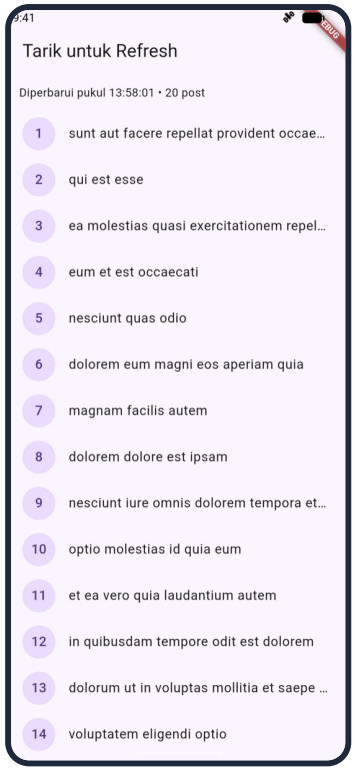
class _RefreshPostScreenState extends State<RefreshPostScreen> {
  final _service = PostService();
  late Future<List<Post>> _future;
  DateTime _terakhir = DateTime.now();

  @override
  void initState() {
    super.initState();
    _future = _service.fetchPosts();
  }

  Future<void> _refresh() async {
    final future = _service.fetchPosts();
    setState(() {
      _future = future;
      _terakhir = DateTime.now();
    });
    try {
      await future; // spinner RefreshIndicator berhenti setelah ini
    } catch (_) {
      // error sudah ditampilkan oleh FutureBuilder, tidak perlu dilempar lagi
    }
  }

  @override
  Widget build(BuildContext context) {
    final jam = '${_terakhir.hour.toString().padLeft(2, '0')}:'
        '${_terakhir.minute.toString().padLeft(2, '0')}:'
        '${_terakhir.second.toString().padLeft(2, '0')}';
    return Scaffold(
      appBar: AppBar(title: const Text('Tarik untuk Refresh')),
      body: RefreshIndicator(
        onRefresh: _refresh,
        child: FutureBuilder<List<Post>>(
          future: _future,
          builder: (context, snapshot) {
            // Loading pertama kali: belum ada data sama sekali
            if (snapshot.connectionState == ConnectionState.waiting && !snapshot.hasData) {
              return const Center(child: CircularProgressIndicator());
            }
            // Error: tetap pakai ListView agar layar masih bisa ditarik
            if (snapshot.hasError) {
```

```
              return ListView(
                physics: const AlwaysScrollableScrollPhysics(),
                children: [
                  const SizedBox(height: 120),
                  const Icon(Icons.cloud_off, size: 64),
                  Center(child: Text('${snapshot.error}')),
                  const Center(child: Text('Tarik ke bawah untuk mencoba lagi')),
                ],
              );
            }
            final posts = snapshot.data ?? [];
            return ListView.builder(
              physics: const AlwaysScrollableScrollPhysics(),
              itemCount: posts.length + 1,
              itemBuilder: (context, index) {
                if (index == 0) {
                  return Padding(
                    padding: const EdgeInsets.all(12),
                    child: Text('Diperbarui pukul $jam • ${posts.length} post'),
                  );
                }
                final post = posts[index - 1];
                return ListTile(
                  leading: CircleAvatar(child: Text('${post.id}')),
                  title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis),
                );
              },
            );
          },
        ),
      ),
    );
  }
}
```



Baris pertama menunjukkan waktu pembaruan terakhir

Penjelasan:

- Kondisi loading memakai `&& !snapshot.hasData`. Saat refresh, Future baru sedang berjalan tetapi kita tidak ingin layar berkedip menjadi spinner besar karena RefreshIndicator sudah menampilkan spinner kecil di atas.
- Tampilan error juga dibuat sebagai `ListView` dengan `AlwaysScrollableScrollPhysics`. Jika memakai `Center` biasa, layar error tidak bisa ditarik untuk mencoba lagi.
- Baris pertama list menampilkan jam pembaruan sehingga efek refresh terlihat jelas.

> ⚠️ **Kesalahan Umum**
>
> - `onRefresh: () { _load(); }` tanpa `async` / `return`: spinner langsung hilang sebelum data datang. Pastikan fungsi mengembalikan Future yang ditunggu.
> - Membungkus `Column` atau `Center` dengan RefreshIndicator: tidak bisa ditarik karena bukan widget scroll.
> - Lupa `AlwaysScrollableScrollPhysics` ketika item sedikit, sehingga gestur tarik tidak berfungsi.

12. Integrasi REST API dengan GetX (Controller & View)

`FutureBuilder` cocok untuk satu layar sederhana. Namun jika data yang sama dipakai banyak layar (daftar, detail, form) dan bisa berubah (tambah/hapus), simpan data di **controller GetX** (Modul 08). Controller menyimpan tiga hal: **isLoading**, **pesan error**, dan **data**. View cukup membaca ketiganya di dalam `Obx`.

Posisi Setiap File dalam Pola MVC

Lapisan dari section 9 tetap dipakai. Dalam pola MVC, `ApiClient` dan `PostRepository` berperan sebagai **service** (akses data); kita cukup menambahkan **controller** dan **view**.

```
lib/
├── main.dart
├── core/
|   ├── api_exception.dart       (section 9)
|   └── api_client.dart          (section 9)
├── models/
|   └── post.dart                (M: section 3)
├── repositories/
|   └── post_repository.dart     (akses data, section 9)
├── controllers/
|   └── post_controller.dart     (C: BARU)
└── views/
    └── post_list_view.dart      (V: BARU)
```

Arah ketergantungan tetap satu arah: **View → Controller → Repository → ApiClient**. Controller tidak tahu soal `http`, dan View tidak tahu soal URL.

Langkah 1: Controller

Pasang package: `flutter pub add get`. Controller menerima `PostRepository` lewat constructor, sehingga saat testing bisa diganti `FakePostRepository`.

`lib/controllers/post_controller.dart`

```dart
import 'package:get/get.dart';

import '../models/post.dart';
import '../repositories/post_repository.dart';

class PostController extends GetxController {
  PostController(this._repo);
  final PostRepository _repo;

  // State privat yang reaktif
  final _posts = <Post>[].obs;
  final _isLoading = false.obs;
  final _error = RxnString(); // String? yang reaktif

  // Getter publik read-only untuk View
  List<Post> get posts => List.unmodifiable(_posts);
  bool get isLoading => _isLoading.value;
```

```
  String? get error => _error.value;

  @override
  void onInit() {
    super.onInit();
    fetchPosts(); // data pertama diambil saat controller dibuat
  }

  Future<void> fetchPosts() async {
    _isLoading.value = true; // View langsung menampilkan loading
    _error.value = null;
    try {
      _posts.assignAll(await _repo.getAll());
    } catch (e) {
      _error.value = e.toString();
    } finally {
      _isLoading.value = false; // View menampilkan data atau error
    }
  }

  /// Optimistic delete: hapus dulu dari layar, kembalikan jika server gagal.
  Future<bool> deletePost(Post post) async {
    final index = _posts.indexOf(post);
    _posts.removeAt(index);
    try {
      await _repo.delete(post.id!);
      return true;
    } catch (e) {
      _posts.insert(index, post); // rollback
      return false;
    }
  }
}
```

Penjelasan:

- Tidak ada pemanggilan manual seperti `update()`. Setiap kali `.value` pada `_isLoading` / `_error` diubah, atau isi `_posts` berubah (`assignAll`, `removeAt`, `insert`), semua `Obx` yang membacanya otomatis dibangun ulang.
- `onInit()` adalah tempat yang tepat untuk request pertama: dipanggil sekali saat controller dibuat oleh `Get.put`.
- `finally` memastikan `isLoading` kembali `false`, baik request berhasil maupun gagal.
- Pada `deletePost` kita memakai pola *optimistic update*: item langsung hilang sehingga aplikasi terasa cepat, lalu dikembalikan jika server menolak. Controller hanya mengembalikan `bool`; yang menampilkan SnackBar adalah View.
- Controller tidak mengimpor `material.dart` dan tidak memegang `BuildContext`, sehingga bisa di-unit-test tanpa widget (Modul 15).

Langkah 2: View

View di-*extend* dari `GetView<PostController>`, sehingga properti `controller` otomatis berisi hasil `Get.find<PostController>()`.

lib/views/post_list_view.dart

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/post_controller.dart';

class PostListView extends GetView<PostController> {
  const PostListView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Obx(() => Text('Post (${controller.posts.length})'))),
      body: Obx(() {
        // 1. Loading
        if (controller.isLoading) {
          return const Center(child: CircularProgressIndicator());
        }
        // 2. Error
        if (controller.error != null) {
          return Center(
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                Text(controller.error!),
                const SizedBox(height: 12),
                FilledButton(
                  onPressed: controller.fetchPosts,
                  child: const Text('Coba Lagi'),
                ),
              ],
            ),
          );
        }
        // 3. Data
        final posts = controller.posts;
        return RefreshIndicator(
          onRefresh: controller.fetchPosts,
          child: ListView.builder(
            itemCount: posts.length,
            itemBuilder: (context, i) {
              final post = posts[i];
              return Dismissible(
                key: ValueKey(post.id),
                direction: DismissDirection.endToStart,
                background: Container(
                  color: Colors.red,
                  alignment: Alignment.centerRight,
                  padding: const EdgeInsets.only(right: 20),
                  child: const Icon(Icons.delete, color: Colors.white),
                ),
                onDismissed: (_) async {
                  final messenger = ScaffoldMessenger.of(context);
                  final ok = await controller.deletePost(post);
                  messenger.showSnackBar(SnackBar(
                    content: Text(ok ? 'Post ${post.id} dihapus' : 'Gagal menghapus, data
dikembalikan'),
                  ));
```

```
                },
                child: ListTile(
                  leading: CircleAvatar(child: Text('${post.id}')),
                  title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis),
                  subtitle: Text(post.body, maxLines: 1, overflow: TextOverflow.ellipsis),
                ),
              );
            },
          ),
        );
      }),
    );
  }
}
```

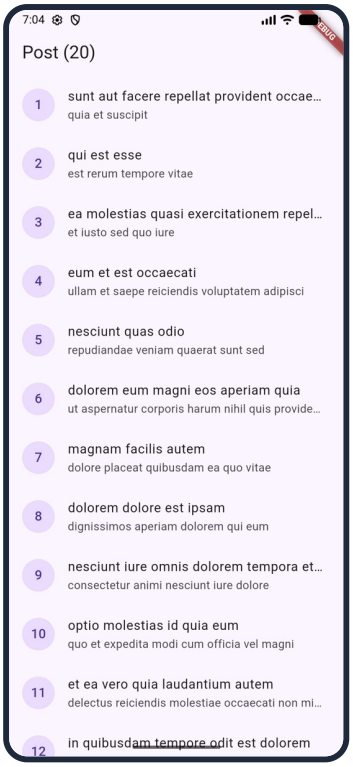
Langkah 3: Daftarkan Controller di `main.dart`

📄 **`lib/main.dart`**

```dart
// Butuh: flutter pub add http get
// File: models/post.dart, core/api_exception.dart, core/api_client.dart,
//       repositories/post_repository.dart, controllers/post_controller.dart,
//       views/post_list_view.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import 'core/api_client.dart';
import 'controllers/post_controller.dart';
import 'repositories/post_repository.dart';
import 'views/post_list_view.dart';

void main() {
  final repo = RemotePostRepository(
    ApiClient(baseUrl: 'https://jsonplaceholder.typicode.com'),
  );
  Get.put(PostController(repo)); // onInit() langsung memanggil fetchPosts()
  runApp(const GetMaterialApp(home: PostListView()));
}
```



Data berhasil dimuat: daftar post dari PostController

Penjelasan:

- `Get.put(PostController(repo))` dipanggil sebelum `runApp`, sehingga controller sudah ada ketika `PostListView` memanggil `Get.find` (lewat `GetView`).
- `Obx` di judul AppBar hanya membaca `posts.length`, jadi hanya judul yang dibangun ulang ketika jumlah post berubah. `Obx` di `body` membaca `isLoading`, `error`, dan `posts`.
- Urutan pengecekan **loading → error → data** menjamin hanya satu kondisi yang tampil.
- Di dalam callback (`onPressed`, `onRefresh`, `onDismissed`) cukup panggil `controller.method()`, tanpa `Obx`.
- `ScaffoldMessenger.of(context)` disimpan ke variabel **sebelum** `await`, sehingga aman dipakai setelahnya.
- Geser item ke kiri untuk menghapus. Karena JSONPlaceholder selalu menjawab sukses, rollback hanya terlihat jika internet dimatikan.

⚠️ **Kesalahan Umum**

- Memanggil `fetchPosts()` di dalam `build()`: request dikirim ulang setiap kali layar dibangun. Panggil di `onInit()` controller.
- Lupa `Get.put` sebelum View dibuka: muncul error *"PostController" not found. You need to call "Get.put(PostController())"*.
- Mengganti list dengan `_posts = hasil`: variabel `final` tidak bisa diisi ulang, dan list reaktifnya hilang. Pakai `_posts.assignAll(hasil)`.
- `Obx` yang tidak membaca satu pun nilai reaktif: GetX melempar error *improper use of a GetX*. Pastikan getter seperti `controller.isLoading` dibaca di dalam fungsi `Obx`, bukan di luar.
- Menampilkan SnackBar atau dialog dari controller. Biarkan controller mengembalikan hasil (`bool`/pesan), lalu View yang bereaksi.

13. Studi Kasus: Aplikasi CRUD Posts Multi-File

Saatnya menggabungkan semua yang sudah dipelajari menjadi satu aplikasi utuh: **Aplikasi Posts** dengan fitur daftar, detail, tambah, edit, dan hapus, memakai ApiClient, Repository, serta pola **MVC dengan GetX** (Model, View, Controller).

Struktur Folder (MVC)

```
lib/
├── main.dart
├── core/
|   ├── api_exception.dart          (dari section 9, tanpa perubahan)
|   └── api_client.dart             (dari section 9, tanpa perubahan)
├── models/
|   └── post.dart                   (M: dari section 3, tanpa perubahan)
├── repositories/
|   └── post_repository.dart        (akses data: dari section 9, tanpa perubahan)
├── controllers/
|   └── post_crud_controller.dart   (C: BARU)
└── views/
    ├── post_list_view.dart         (V: BARU)
    ├── post_detail_view.dart       (V: BARU)
    └── post_form_view.dart         (V: BARU, dipakai untuk tambah dan edit)
```

Persiapan: `flutter pub add http get`, lalu salin empat file "dari section" di atas. Kita hanya menulis file yang BARU.

> ℹ️ **Menyiasati JSONPlaceholder**
>
> Karena server palsu selalu memberi `id: 101` untuk post baru dan tidak menyimpan perubahan, controller kita (1) memberi id unik lokal untuk post baru, dan (2) tidak mengirim PUT untuk post buatan sendiri (id > 100) karena server akan membalas 500. Di backend sungguhan, dua siasat ini tidak diperlukan.

1. Controller CRUD

📄 **`lib/controllers/post_crud_controller.dart`**

```dart
import 'package:get/get.dart';

import '../models/post.dart';
import '../repositories/post_repository.dart';

class PostCrudController extends GetxController {
  PostCrudController(this._repo);
  final PostRepository _repo;
```

```dart
final _posts = <Post>[].obs;
final _isLoading = false.obs;
final _error = RxnString();
final _saving = false.obs; // true saat POST/PUT/DELETE berjalan

List<Post> get posts => List.unmodifiable(_posts);
bool get isLoading => _isLoading.value;
String? get error => _error.value;
bool get saving => _saving.value;

Post? findById(int id) {
  for (final p in _posts) {
    if (p.id == id) return p;
  }
  return null;
}

@override
void onInit() {
  super.onInit();
  load(); // READ pertama saat controller dibuat
}

// READ
Future<void> load() async {
  _isLoading.value = true;
  _error.value = null;
  try {
    _posts.assignAll(await _repo.getAll());
  } catch (e) {
    _error.value = '$e';
  } finally {
    _isLoading.value = false;
  }
}

// CREATE
Future<void> add(String title, String body) async {
  await _runSaving(() async {
    final created = await _repo.create(Post(userId: 1, title: title, body: body));
    // JSONPlaceholder selalu memberi id 101, jadi buat id unik lokal
    final nextId = _posts.fold<int>(100, (max, p) => p.id! > max ? p.id! : max) + 1;
    _posts.insert(0, created.copyWith(id: nextId));
  });
}

// UPDATE
Future<void> edit(Post post) async {
  await _runSaving(() async {
    // Post buatan sendiri (id > 100) tidak dikenal server palsu, cukup ubah lokal
    final saved = post.id! > 100 ? post : await _repo.update(post);
    final i = _posts.indexWhere((p) => p.id == post.id);
    if (i != -1) _posts[i] = saved;
  });
}

// DELETE
Future<void> remove(int id) async {
  await _runSaving(() async {
```

```
      if (id <= 100) await _repo.delete(id);
      _posts.removeWhere((p) => p.id == id);
    });
  }

  // Pembantu: set flag saving, jalankan aksi, lempar ulang error ke View
  Future<void> _runSaving(Future<void> Function() action) async {
    _saving.value = true;
    try {
      await action();
    } finally {
      _saving.value = false;
    }
  }
}
```

Penjelasan: method `add`, `edit`, dan `remove` tidak menangkap error. Error dibiarkan naik ke View form/detail agar View itu bisa menampilkan SnackBar dan **tidak menutup** form jika gagal. `try/finally` di `_runSaving` memastikan flag `saving` kembali `false` walaupun terjadi error. Perhatikan juga bahwa controller tidak bernavigasi dan tidak menampilkan SnackBar; semua itu tugas View.

2. View Daftar

`lib/views/post_list_view.dart`

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/post_crud_controller.dart';
import 'post_detail_view.dart';
import 'post_form_view.dart';

class PostListView extends GetView<PostCrudController> {
  const PostListView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Aplikasi Posts')),
      body: Obx(() {
        if (controller.isLoading) {
          return const Center(child: CircularProgressIndicator());
        }
        if (controller.error != null) {
          return Center(
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                const Icon(Icons.cloud_off, size: 56),
                const SizedBox(height: 8),
                Text(controller.error!),
                TextButton(onPressed: controller.load, child: const Text('Coba Lagi')),
              ],
            ),
          );
```

```
        }
        final posts = controller.posts;
        if (posts.isEmpty) {
          return const Center(child: Text('Belum ada post.'));
        }
        return RefreshIndicator(
          onRefresh: controller.load,
          child: ListView.separated(
            physics: const AlwaysScrollableScrollPhysics(),
            itemCount: posts.length,
            separatorBuilder: (_, _) => const Divider(height: 1),
            itemBuilder: (context, i) {
              final post = posts[i];
              return ListTile(
                leading: CircleAvatar(child: Text('${post.id}')),
                title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis),
                subtitle: Text(post.body, maxLines: 2, overflow: TextOverflow.ellipsis),
                onTap: () => Get.to(() => PostDetailView(postId: post.id!)),
              );
            },
          ),
        );
      }),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => Get.to(() => const PostFormView()),
        icon: const Icon(Icons.add),
        label: const Text('Post Baru'),
      ),
    );
  }
}
```

Penjelasan: `Get.to(() => HalamanBaru())` adalah pengganti `Navigator.push` di GetX; tidak butuh `context`. Halaman detail dan form tidak perlu menerima controller lewat constructor, karena keduanya cukup memanggil `Get.find` (lewat `GetView`) untuk mendapatkan **instance yang sama**.

3. View Detail (dengan Edit & Hapus)

`lib/views/post_detail_view.dart`

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/post_crud_controller.dart';
import 'post_form_view.dart';

class PostDetailView extends GetView<PostCrudController> {
  final int postId;
  const PostDetailView({super.key, required this.postId});

  Future<void> _konfirmasiHapus(BuildContext context) async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (ctx) => AlertDialog(
        title: const Text('Hapus post?'),
        content: const Text('Post yang dihapus tidak bisa dikembalikan.'),
        actions: [
```

```
          TextButton(onPressed: () => Navigator.pop(ctx, false), child: const Text('Batal')),
          FilledButton(onPressed: () => Navigator.pop(ctx, true), child: const Text('Hapus')),
        ],
      ),
    );
    if (yakin != true || !context.mounted) return;

    final messenger = ScaffoldMessenger.of(context);
    try {
      await controller.remove(postId);
      Get.back(); // kembali ke daftar
      messenger.showSnackBar(const SnackBar(content: Text('Post dihapus')));
    } catch (e) {
      messenger.showSnackBar(SnackBar(content: Text('Gagal menghapus: $e')));
    }
  }

  @override
  Widget build(BuildContext context) {
    return Obx(() {
      final post = controller.findById(postId);

      // Post bisa saja sudah terhapus ketika layar ini masih terbuka
      if (post == null) {
        return Scaffold(appBar: AppBar(), body: const Center(child: Text('Post tidak ditemukan')));
      }

      return Scaffold(
        appBar: AppBar(
          title: Text('Post #${post.id}'),
          actions: [
            IconButton(
              tooltip: 'Edit',
              icon: const Icon(Icons.edit),
              onPressed: () => Get.to(() => PostFormView(post: post)),
            ),
            IconButton(
              tooltip: 'Hapus',
              icon: const Icon(Icons.delete_outline),
              onPressed: controller.saving ? null : () => _konfirmasiHapus(context),
            ),
          ],
        ),
        body: ListView(
          padding: const EdgeInsets.all(20),
          children: [
            Text(post.title, style: Theme.of(context).textTheme.headlineSmall),
            const SizedBox(height: 4),
            Text('Penulis: user ${post.userId}', style: Theme.of(context).textTheme.labelMedium),
            const Divider(height: 32),
            Text(post.body, style: Theme.of(context).textTheme.bodyLarge),
          ],
        ),
      );
    });
  }
}
```

Penjelasan: seluruh halaman dibungkus `Obx` karena isinya bergantung pada `findById` (yang membaca list reaktif) dan `saving`. Setelah post diedit, halaman detail otomatis menampilkan judul baru tanpa perlu dikirimi data ulang.

4. Satu Form untuk Tambah dan Edit

Jika parameter `post` bernilai `null`, form berada dalam mode **tambah**; jika diisi, mode **edit** dan field terisi data lama. `TextEditingController` adalah state lokal milik form, jadi tetap disimpan di `StatefulWidget`; controller GetX hanya dipakai untuk data post.

`lib/views/post_form_view.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/post_crud_controller.dart';
import '../models/post.dart';

class PostFormView extends StatefulWidget {
  final Post? post; // null = tambah, ada isi = edit
  const PostFormView({super.key, this.post});

  @override
  State<PostFormView> createState() => _PostFormViewState();
}

class _PostFormViewState extends State<PostFormView> {
  final _controller = Get.find<PostCrudController>();
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _titleCtrl;
  late final TextEditingController _bodyCtrl;

  bool get _isEdit => widget.post != null;

  @override
  void initState() {
    super.initState();
    _titleCtrl = TextEditingController(text: widget.post?.title ?? '');
    _bodyCtrl = TextEditingController(text: widget.post?.body ?? '');
  }

  @override
  void dispose() {
    _titleCtrl.dispose();
    _bodyCtrl.dispose();
    super.dispose();
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    final title = _titleCtrl.text.trim();
    final body = _bodyCtrl.text.trim();

    try {
      if (_isEdit) {
        await _controller.edit(widget.post!.copyWith(title: title, body: body));
      } else {
```

```
        await _controller.add(title, body);
      }
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(_isEdit ? 'Perubahan disimpan' : 'Post ditambahkan')),
      );
      Get.back();
    } catch (e) {
      if (!mounted) return;
      // Form TIDAK ditutup agar isian pengguna tidak hilang
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Gagal: $e')));
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(_isEdit ? 'Edit Post' : 'Post Baru')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _titleCtrl,
              decoration: const InputDecoration(labelText: 'Judul', border: OutlineInputBorder()),
              validator: (v) => (v == null || v.trim().length < 3) ? 'Judul minimal 3 karakter' :
null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _bodyCtrl,
              maxLines: 5,
              decoration: const InputDecoration(labelText: 'Isi', border: OutlineInputBorder()),
              validator: (v) => (v == null || v.trim().isEmpty) ? 'Isi tidak boleh kosong' : null,
            ),
            const SizedBox(height: 20),
            // Hanya tombol ini yang dibangun ulang saat saving berubah
            Obx(() {
              final saving = _controller.saving;
              return FilledButton.icon(
                onPressed: saving ? null : _simpan,
                icon: saving
                    ? const SizedBox(width: 18, height: 18, child:
CircularProgressIndicator(strokeWidth: 2))
                    : const Icon(Icons.save),
                label: Text(saving ? 'Menyimpan...' : 'Simpan'),
              );
            }),
          ],
        ),
      ),
    );
  }
}
```

5. `main.dart`

`lib/main.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import 'controllers/post_crud_controller.dart';
import 'core/api_client.dart';
import 'repositories/post_repository.dart';
import 'views/post_list_view.dart';

/// Mendaftarkan semua dependency aplikasi di satu tempat.
class AppBinding extends Bindings {
  @override
  void dependencies() {
    final api = ApiClient(baseUrl: 'https://jsonplaceholder.typicode.com');
    Get.put(PostCrudController(RemotePostRepository(api)), permanent: true);
  }
}

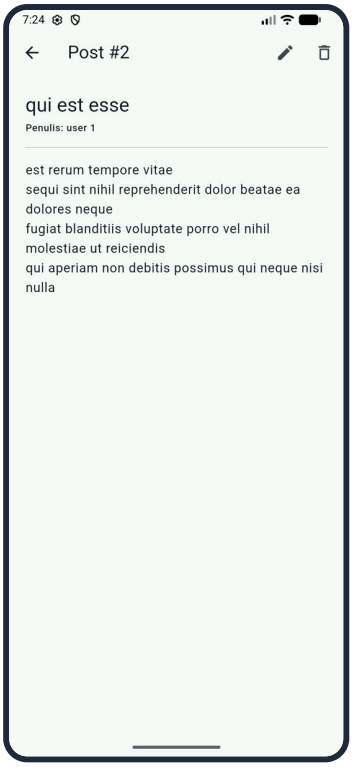
void main() {
  runApp(const PostsApp());
}

class PostsApp extends StatelessWidget {
  const PostsApp({super.key});

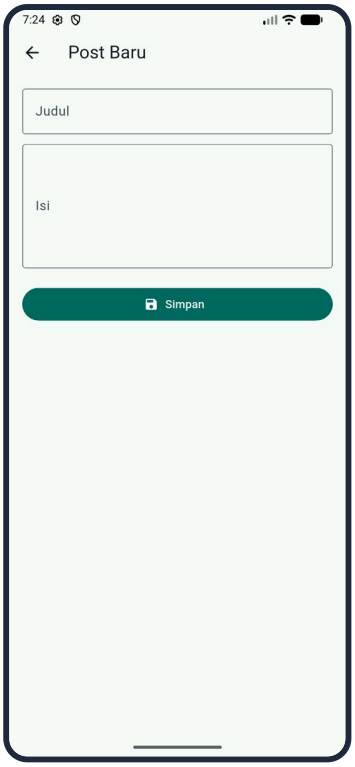
  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      title: 'Aplikasi Posts',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      initialBinding: AppBinding(),
      home: const PostListView(),
    );
  }
}
```



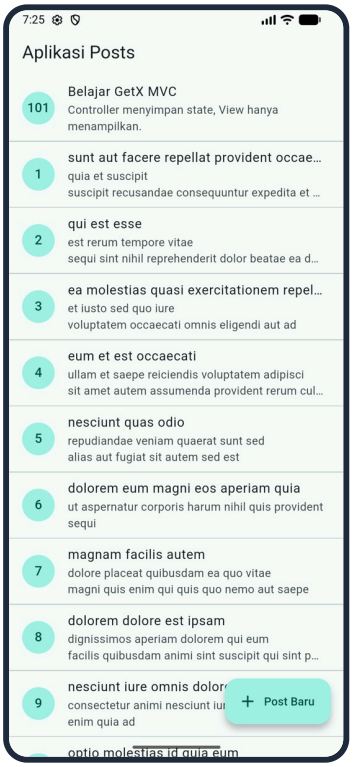
Layar daftar



Layar detail dengan tombol edit dan hapus



Form tambah post



Setelah disimpan: post baru muncul paling atas

Penjelasan: `AppBinding` dijalankan oleh `GetMaterialApp` sebelum halaman pertama dibangun, sehingga `PostListView` sudah bisa menemukan `PostCrudController`. `permanent: true` menjaga controller tetap hidup selama aplikasi berjalan, karena dipakai oleh semua halaman.

Alur Data Saat Menambah Post

```
PostFormView._simpan()
   └─> PostCrudController.add()        saving.value = true (Obx tombol: "Menyimpan...")
         └─> RemotePostRepository.create()
               └─> ApiClient.post('/posts', json)   -> 201 Created
         <- Post baru di-insert ke _posts (RxList), saving.value = false
   <- SnackBar "Post ditambahkan", Get.back()
PostListView (Obx membaca controller.posts) otomatis rebuild, post baru muncul paling atas
```

> ⚠️ **Kesalahan Umum pada Proyek CRUD**
>
> - Menutup form (`Get.back()`) sebelum `await` selesai, sehingga error tidak pernah terlihat.
> - Mengirim objek yang sama ke halaman detail lalu mengeditnya: detail tidak berubah. Kirim `id` lalu cari data terbaru dari controller (seperti `findById`).
> - Menekan tombol Simpan berkali-kali sehingga data dobel. Nonaktifkan tombol saat `saving` .
> - Memakai `context` setelah `await` tanpa cek `mounted` / `context.mounted` .
> - Membuat controller baru di setiap halaman (`Get.put(PostCrudController(...))` di detail/form). Data jadi tidak sinkron; cukup `Get.find` controller yang sudah didaftarkan di `AppBinding` .
> - Memanggil `Get.back()` atau menampilkan SnackBar dari dalam controller. Controller cukup melempar error atau mengembalikan hasil; View yang memutuskan navigasi dan pesan.

Tugas Praktikum: Praktikum 9: Aplikasi Katalog Tugas Kuliah Berbasis REST API

Sasaran Praktikum

Membangun aplikasi CRUD yang terhubung ke REST API publik dengan arsitektur berlapis dan pola MVC (ApiClient, Repository, Controller GetX, View) dan penanganan error yang ramah pengguna.

Langkah Pengerjaan

1. Gunakan endpoint `https://jsonplaceholder.typicode.com/todos` (atau mock API pribadi seperti mockapi.io agar data benar-benar tersimpan).
2. Buat model `Todo` (id, userId, title, completed) lengkap dengan `fromJson`, `toJson`, dan `copyWith`.
3. Buat `ApiClient` yang menambahkan header JSON, memberi timeout 10 detik, dan mengubah semua kegagalan menjadi `ApiException`.
4. Buat `TodoRepository` (abstract) beserta `RemoteTodoRepository` dan `FakeTodoRepository`.
5. Buat `TodoController` (GetxController) dengan state reaktif `isLoading`, `error`, dan daftar todo; muat data di `onInit()` dan daftarkan dengan `Get.put` atau `Bindings`. Tampilkan daftar todo dengan `?_limit=20` dan pull-to-refresh memakai `Obx`.
6. Susun folder `models/`, `repositories/`, `controllers/`, dan `views/`. Sediakan view detail, form tambah/edit (satu form untuk dua mode, dengan validasi), dan hapus dengan dialog konfirmasi; navigasi memakai `Get.to` / `Get.back`.
7. Ubah status selesai todo memakai request `PATCH` yang hanya mengirim field `completed`.
8. Uji mode pesawat dan host yang salah: aplikasi wajib menampilkan pesan error dan tombol coba lagi, bukan crash. Tambahkan izin INTERNET lalu uji juga dengan build release.

Output yang Diharapkan

Aplikasi daftar tugas yang memuat data dari internet, bisa menambah, mengedit, menandai selesai, dan menghapus data dengan umpan balik visual (loading, SnackBar, pesan error) yang jelas.

Rubrik Penilaian

Model & parsing JSON (15%), ApiClient + error handling (20%), Repository & Controller GetX (20%), Operasi CRUD + PATCH berjalan (25%), UX: loading, pull-to-refresh, validasi, konfirmasi hapus (20%).

Kuis Pemahaman

1. Mengapa variabel 'Future' pada FutureBuilder sebaiknya diinisialisasi di dalam method 'initState()', bukan langsung dipanggil di dalam method 'build()'?

A. Agar request HTTP tidak terpanggil berulang kali setiap kali layar berkedip atau di-rebuild oleh framework

B. Karena method build() tidak mendukung internet

C. Agar ukuran aplikasi menjadi lebih kecil

D. Supaya status HTTP otomatis bernilai 200

2. Kode status HTTP (Status Code) manakah yang mengindikasikan bahwa data baru BERHASIL dibuat di server?

A. 200 OK

B. 201 Created

C. 404 Not Found

D. 500 Internal Server Error

3. Apa hasil jsonDecode('["a", "b"]') di Dart?

A. List<dynamic>

B. List<String>

C. Map<String, dynamic>

D. String

4. Request http.get mengembalikan status 404. Apa yang terjadi pada kode yang hanya memakai try/catch tanpa mengecek statusCode?

A. Masuk ke blok catch secara otomatis

B. Tidak ada exception; kode melanjutkan seolah sukses sehingga harus mengecek statusCode sendiri

C. Aplikasi langsung tertutup

D. Package http mengulang request secara otomatis

5. Apa perbedaan utama PUT dan PATCH?

A. PUT untuk membuat data, PATCH untuk menghapus

B. PUT mengganti seluruh data, PATCH hanya mengubah field yang dikirim

C. PATCH lebih aman dari PUT

D. Tidak ada perbedaan

6. Format header yang benar untuk mengirim token login adalah...

A. 'Token': token

B. 'Authorization': 'Bearer $token'

C. 'Authorization': 'Bearer:$token'

D. 'Bearer': token

7. Mengapa body POST harus di-jsonEncode dan diberi header Content-Type: application/json?

A. Agar request lebih cepat

B. Karena tanpa itu package http mengirim Map sebagai form-urlencoded dan server tidak membacanya sebagai JSON

C. Karena Dart tidak mendukung Map

D. Agar status code menjadi 200

8. Pada RefreshIndicator, mengapa daftar perlu physics: AlwaysScrollableScrollPhysics()?

A. Agar scroll lebih cepat

B. Agar gestur tarik tetap bisa dilakukan walaupun isi list sedikit atau sedang menampilkan pesan error

C. Agar list tidak bisa di-scroll

D. Karena RefreshIndicator mewajibkan GridView

Kunci Jawaban & Pembahasan

1. A — Jika memanggil fungsi API langsung di dalam method build(), setiap kali ada interaksi kecil (seperti mengetik di keyboard atau animasi), build() dipanggil ulang, memicu pengiriman request HTTP berulang kali ke server tanpa henti.

2. B — Dalam standar arsitektur REST API, kode 201 Created menandakan bahwa permintaan POST untuk membuat entitas baru berhasil dieksekusi oleh server.

3. A — Array JSON selalu di-decode menjadi List<dynamic>. Untuk mendapatkan List<String> gunakan List<String>.from(...) atau .cast<String>().

4. B — Package http hanya melempar exception untuk kegagalan koneksi. Status 4xx/5xx tetap dianggap response yang valid, sehingga kita wajib mengecek statusCode.

5. B — PUT mengirim representasi lengkap objek sebagai pengganti, sedangkan PATCH hanya mengirim field yang berubah.

6. B — Standar Bearer token memakai header Authorization dengan nilai 'Bearer ' (ada spasi) diikuti token.

7. B — Jika body berupa Map, http mengirimnya sebagai form. jsonEncode mengubahnya menjadi teks JSON dan Content-Type memberi tahu server cara membacanya.

8. B — Jika konten tidak memenuhi layar, ListView tidak bisa di-scroll sehingga gestur pull-to-refresh tidak terpicu. AlwaysScrollableScrollPhysics memaksa list tetap bisa ditarik.